Sustain

How the application works from the first screen to the last database write

A practical implementation and system-design guide for a developer learning to build, explain and operate an iPhone-first health-tracking application.

Reviewed source: `4fb1bddb1d95af71564828cc3cf8d202bf12ae7b`
Review and documentation date: 25 September 2026
Scope: Current repository implementation, synthetic verification, engineering tradeoffs and a measured path to release.

This guide uses invented examples. No example account, reading, meal or credential belongs to a real person. Sample identifiers such as `U1` are teaching abbreviations, not valid UUIDs to send to the application.

A little care. Every day.

Learn the system by following its data.

Contents

Follow the implementation first, then use the decision and interview chapters to test your reasoning. Page numbers refer to this PDF.

01. Purpose, scope and how to read this guide 4

02. How React Native turns code into screens 6

03. The complete system and its boundaries 8

04. Calls, APIs and the journey of one fluid entry 11

05. Building and evolving the database 15

06. The actual tables, indexes and database programs 18

07. Signup, sessions, onboarding and secure entry 22

08. Face ID: unlock, enrollment and credential exchange 24

09. Reliable writes: one meal, one identity, one receipt 27

10. Offline vitals, outboxes and synchronization cursors 30

11. Apple Health, from native sample to displayed reading 33

12. Lifting, cardio and exercise guidance 38

13. Food labels, meal snapshots and reusable recipes 40

14. Barcode lookup and external product data 42

15. AI meal estimates and workout plans 44

16. Fluids, goals and calculations that preserve their basis 47

Contents, continued

Follow the implementation first, then use the decision and interview chapters to test your reasoning. Page numbers refer to this PDF.

17. Summary, charts, History and derived streaks 49

18. Reminders: local scheduling and safe navigation 52

19. Progress photos and the boundary of a transaction 54

20. Export: collecting, packaging and sharing account data 56

21. Account deletion as a resumable protocol 58

22. Settings, privacy boundaries and operational ownership 60

23. Design choices, credible alternatives and when to change 62

24. From source code to a released iPhone app 67

25. Scaling and cost without speculative infrastructure 69

26. Verification, known gaps and an improvement plan 71

27. Security and distributed-systems questions behind the features 73

28. Interview answers with deeper follow-up chains 77

29. Follow-up drills, counterexamples and whiteboard practice 82

30. Working glossary 86

31. Source map, evidence and verification boundaries 88

01. Purpose, scope and how to read this guide

Sustain began as an application for its owner's everyday use. The stated objective was useful functionality that one person could develop and maintain, with room to add users later. Learning broadly useful technologies, particularly the React ecosystem and TypeScript, was also an explicit motivation. These goals favor a small operational footprint, understandable code and targeted reliability work. They do not require a distributed platform designed for an assumed million users.

Even one user encounters difficult failures. A phone loses connectivity after the server accepts a meal. The operating system terminates the process while a draft is open. A second device edits an old value. A notification arrives while the app is locked. A photo uploads but its database row is rejected. These are everyday correctness problems, not problems reserved for large companies. Much of Sustain's complexity protects against them.

The current product combines manual and Apple Health vitals, lifting and cardio logs, nutrition and reusable recipes, fluids, goals, Summary widgets, history, local reminders, optional AI assistance, photos, export and account controls. It is a wellness tracker. A stored measurement, estimated calorie target or model-generated plan is not a clinical assessment.

Four different questions

First ask whether a feature exists in the current source and can be reached from the interface. Then ask whether it has been tested under a particular condition. Separately ask whether the corresponding server version is deployed. Finally ask whether the installed native app behaves correctly on an actual iPhone. A passing browser test answers only part of the second question.

This guide uses these evidence categories consistently:

Category	Meaning in this report
Current implementation	The reviewed files and connected execution path implement the behavior.
Verified in this review	A named check actually ran against this source snapshot.
Historical record	Repository documentation reports a previous deployment, build or test. It was not repeated here.
Proposed improvement	An engineering recommendation, not existing behavior.
Unknown or unverified	Available code and checks do not establish the claim.

The review started from a clean working tree at the commit on the cover. Documentation from the preceding review is the only preexisting working-tree change during authoring. No application code or tests were changed for this guide. No real account was queried or modified, no paid AI request was made and no service was deployed. There is no fresh independently verified hosted-schema claim in this report.

Learn in execution order

Read Chapters 2-6 for the programming, network and database foundations. Chapters 7-10 explain identity, retries and synchronization. The feature chapters then follow actual calls from user action to visible result. The later chapters compare designs, explain deployment and growth, and prepare interview answers that depend on the preceding teaching.

A **call** means one piece of code asking another piece to perform work. Some calls finish in the same JavaScript process. Others cross into iOS, access durable device storage or travel over a network. Always locate the boundary: who owns the data, what can fail independently and what result is actually known? This habit is more useful than memorizing technology names.

Source labels such as **[C08]** identify repository paths and functions in the evidence appendix. **[R03]** identifies an official external reference. Examples are labeled as actual excerpts, source-shaped synthetic objects or simplified pseudocode. Omitted fields are not permission to assume the abbreviated example is a complete production request.

What the review established

Lint, TypeScript, 256 application tests and 70 Edge Function tests passed. The maintained isolated PostgreSQL harness applied all 37 migrations and passed its security, setup, streak, privacy and deletion regressions. The complete synthetic browser suite passed 110 of 112 tests. Signup focus and repeated diagonal chart gestures failed and failed again in focused unchanged reruns. Those results remain visible in this guide; a partially passing suite is not described as fully passing.

Physical-iPhone checks, genuine multi-connection database concurrency, deployed privacy-contract compatibility, provider contracts and production operations remain separate work. Supporting multiple `user_id` values demonstrates an account model, not a measured capacity. The architecture is understandable enough to extend, but its release and scale claims must stay proportional to evidence. [C01, C30]

02. How React Native turns code into screens

An input form has two jobs: remember what the user typed and display a view corresponding to those values. In a traditional imperative interface, code might locate a text label and change it directly. React instead asks the developer to describe what the view should look like for a given state. When state changes, React runs the component again and reconciles the resulting description with the existing interface.

A **component** is a function that produces an interface description. Sustain combines React Native components such as `View`, `Text` and `TextInput` with shared app components such as fields, pressable controls and sheets. React Native connects that description to platform views. Browser preview uses web implementations of the same app structure; it does not turn a browser into an iPhone.

State, hooks and controlled inputs

State is information React remembers between renders and uses to decide what to display. A controlled input receives its displayed text from state and calls a handler when the user types. Keeping the raw text matters: an empty string and the number zero are different while a person edits a workout weight.

The following is teaching pseudocode, not a copied Sustain component. It illustrates the structure used by the forms:

```
const [amountText, setAmountText] = useState("");
const [saving, setSaving] = useState(false);
const [error, setError] = useState<string>();

<TextInput value={amountText} onChangeText={setAmountText} />
<Button disabled={saving} onPress={save} />
{error ? <Text>{error}</Text> : null}
```

`useState` is a **hook**: a React function that attaches remembered behavior to a component. Calling `setAmountText("250")` schedules a new render with that text. It does not save 250 to a database. Calling `setSaving(true)` makes the button reflect work in progress. That visual state is separate from the operation's durable identity, which later chapters explain.

An **effect**, commonly created with `useEffect`, runs work connected to a component's lifetime or changing inputs. Sustain uses effects to subscribe to app foreground events, restore drafts and load resources. Its cleanup function removes subscriptions or invalidates obsolete work. Otherwise a departed screen could still update state or an old account request could publish data into a new account's interface.

A **ref** created with `useRef` holds a mutable value without itself triggering a render. Refs are useful for the latest account identity, a request generation number or an immediate submission guard. A state update is scheduled; a ref can be set synchronously before a second tap arrives. Neither survives process termination. That is why refs solve rapid double taps but cannot solve lost-response retries across restarts.

Providers share an authority, not every field

A React **context provider** makes a value available to components beneath it. AuthProvider supplies the current session and security state; the appearance provider supplies the resolved color scheme. A nested screen can call `useAuth()` instead of receiving the session through every intermediate component.

The root layout composes gesture handling, appearance, motion, authentication, deletion recovery, account setup and entry coordination. Order matters. A setup provider needs an authenticated identity; protected navigation needs both. A provider is not automatically durable storage. AuthProvider's memory and account setup's cloud row have different lifetimes. [C02, C03]

Expo Router maps files in `app/` to routes. Parent `_layout.tsx` files wrap their children in navigators and gates. A navigation call selects a destination; it does not independently authorize it. The protected app layout checks authentication and setup, and the privacy boundary can cover already-mounted tabs. The raised Track control opens a modal menu with six logging destinations. Soon is currently a placeholder rather than an AI chat tab.

Why domain code and repositories are separate

A calculation such as ounces to milliliters needs numbers, not a keyboard or navigation stack. Keeping it in a domain/model module permits direct tests with known inputs. A **repository** is the feature's group of read/write functions. For example, `saveHydration` knows which fields the cloud expects, while the screen knows which message and controls to show. This avoids placing authentication filters, conversions and network response parsing in every button handler.

A **platform adapter** implements the same responsibility differently on each platform. Vitals use SQLite on native and IndexedDB in a browser. Export shares a temporary file on native and downloads a Blob in the browser. HealthKit's iOS reader calls the native module; the fallback cannot read Apple Health. Platform-suffixed files let the build select the suitable implementation without scattering every platform condition through every screen.

Shared theme and appearance modules keep colors, spacing and typography coherent. Shared sheets handle bounded scrolling and focus behavior. This reduces repeated fixes, but also means a defect in a shared control can affect many screens. Browser accessibility checks and native VoiceOver checks therefore complement one another rather than substitute for one another. [C02, C29]

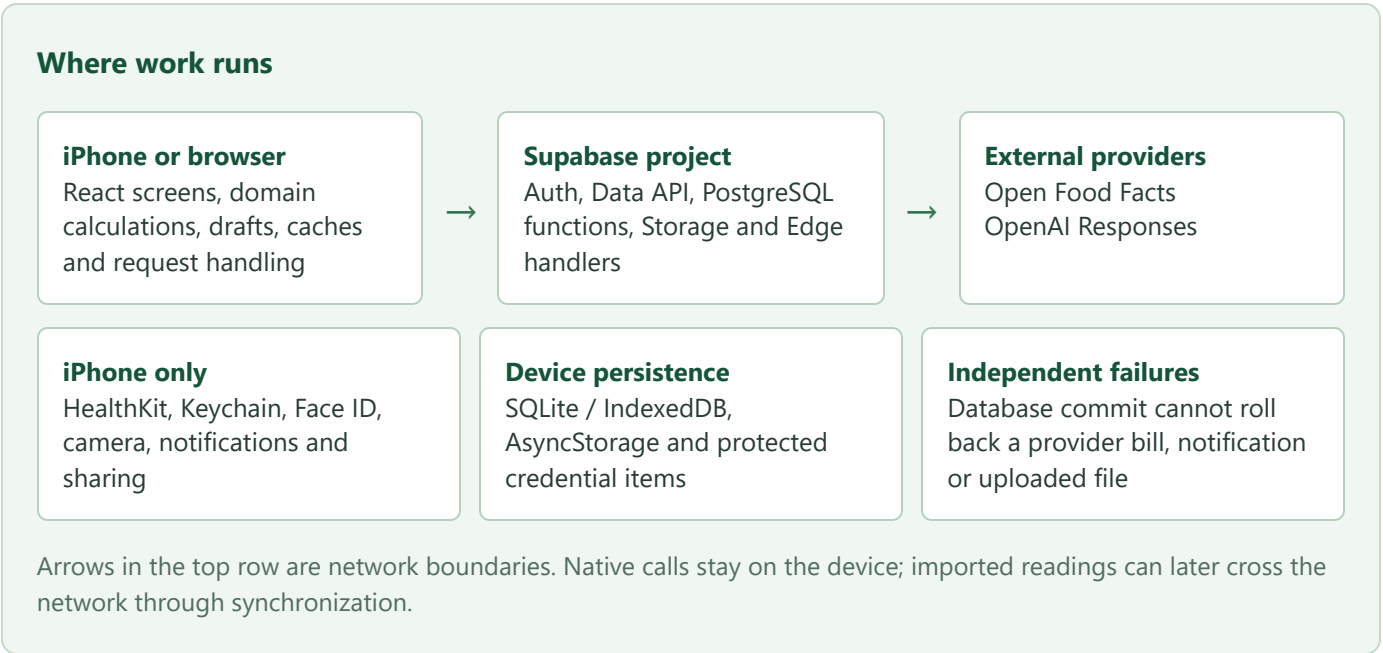
A teaching construction sequence

To rebuild a simplified tracker, first create navigation and one local form. Define the input model and a pure conversion function. Add authentication and a user-owned table. Connect one save and one read, then show loading and errors without discarding entered text. Add server constraints and cross-account tests. Introduce operation receipts once an uncertain retry could duplicate records. Add an outbox only if the feature must accept durable offline writes.

This is a suggested learning sequence, not Sustain's historical build order. It deliberately lets each new mechanism solve a visible problem. Building a generic synchronization engine before a single form works makes its requirements harder to see. Conversely, shipping a health log whose only protection is a disabled Save button leaves a real correctness gap even for one user.

03. The complete system and its boundaries

The installed app and the hosted backend are different programs with different lifetimes. The app renders screens, performs local calculations, stores selected device state and calls native services. Supabase hosts authentication, a PostgreSQL database, a generated data API, object storage and Edge Functions. Open Food Facts supplies food-product information. OpenAI supplies optional model inference through server requests.



Arrows in the top row are network boundaries. Native calls stay on the device; imported readings can later cross the network through synchronization.

What Supabase supplies and what Sustain still owns

Without a managed backend, the developer would need to host PostgreSQL, configure access, create a public API, operate authentication endpoints, manage file storage and deploy server code for provider secrets. Supabase packages those services. Its JavaScript SDK constructs requests and parses responses; its Data API exposes permitted tables and functions over HTTP.

Sustain still owns its schema, access policies, input rules, migrations, provider usage limits, deletion protocol, backups/restoration practice and release compatibility. A managed database cannot infer whether one workout set belongs to another user's session. A hosted authentication service cannot infer that a prepared account deletion should block future application writes. Those rules are in Sustain's SQL and handlers.

The **client** is the calling app. A **server** is a remote process that responds. PostgreSQL stores durable structured records. Edge Functions are server programs executed in a hosted runtime, useful when code must hold a secret, call a provider or coordinate account operations. The name does not imply every app request passes through an Edge Function. Most ordinary reads use the Data API directly; hardened record writes commonly call a PostgreSQL function; AI requests invoke Edge handlers. [C04, C05, C07]

Four storage lifetimes

In-memory React state lasts while that process and relevant component remain alive. AsyncStorage drafts and SQLite/IndexedDB vitals survive normal process restarts. SecureStore/Keychain protects selected native credential material under operating-system rules. Cloud rows and object files survive removal of the app unless separately deleted.

These stores do not form one automatically synchronized database. Changing a local appearance preference does not update a cloud profile. Saving a recipe to Supabase does not ensure a pending meal draft was cleared. A ZIP shared to another application becomes a separate copy beyond Sustain's cleanup control.

Information	Principal home	Consequence
Current form text	React state, with drafts for selected forms	Unsaved ephemeral fields can disappear on termination.
Vitals cache and outbox	Account-scoped SQLite or IndexedDB	Can retain pending manual readings while offline.
Goals and completed logs	Supabase PostgreSQL	Multiple devices can read an account's accepted records.

Information	Principal home	Consequence
Progress photo bytes	Private Storage bucket	Metadata relationships and file cleanup need coordination.
Reminder definitions and widget layout	Account-scoped local storage	Preferences differ between devices unless explicitly exported.
Face ID sign-in secret	Authenticated native Keychain item	Device verification and recovery differ from ordinary drafts.

The app's public configuration includes a Supabase project URL and publishable key. A distributed client cannot keep these secret. They identify the project and API access context; account permissions still depend on the user's token and server policies. Service-role keys and provider API keys belong in server configuration because they grant capabilities beyond an ordinary user. Putting them in a mobile bundle would distribute them to everyone who installs it. [C04, C23]

Reachability and compatibility

Repositories contain legacy types and server structures from earlier product directions. Coach threads still exist, but the current planner does not expose a general conversation browser. HealthKit mapping helpers mention activities, but the active import path reads vitals rather than workouts. The presence of a type, table or helper is evidence of code, not evidence of a current feature.

An installed app is also a versioned client of a remotely changing service. The current repository includes privacy-consent changes that historical STATUS says are not deployed. Running that client against an older server can produce missing-function/table errors or different enforcement. The deployment chapter treats client/server compatibility as part of correctness, not merely packaging.

04. Calls, APIs and the journey of one fluid entry

An **application programming interface**, or API, is an agreed way to ask software to do something. A local function API can be as simple as `hydrationAmountToMl(8, "fl_oz")`. A network API encodes a request, sends it to a remote address and waits for a response. A native-library API asks the operating system to do work through a compiled bridge.

An HTTP request has a method such as GET or POST, a destination URL, headers and sometimes a body. Headers carry metadata such as content type and authorization. A JSON body carries structured values. The server returns a status code and usually a body. A rejected request and a missing response are different: rejection can establish that no accepted write occurred, while a lost response leaves the client uncertain.

What the SDK hides

A **software development kit**, or SDK, packages repeated protocol work. Supabase's SDK builds URLs, encodes filters and JSON, attaches the configured project key and current session bearer token, and returns a result containing `data` and `error`. It does not know whether the interface should preserve a draft, whether a version conflict should prompt a review, or whether a second save means a retry or a new meal. Sustain implements those decisions.

The following map distinguishes code-visible calls from their network behavior. Endpoint shapes are SDK/service conventions verified against the dependency and API patterns, not literal URL strings written at every call site. `<project>` and angle-bracketed values are placeholders.

Code-visible call	Boundary and operation
`hydrationAmountToMl(...)`	Ordinary in-process calculation; no network or permission prompt.
`AsyncStorage.setItem(key, text)`	Durable device/browser key-value storage via a library.
`healthKit.queryQuantitySamplesWithAnchor(...)`	iPhone native HealthKit query; no web endpoint.
`supabase.from("hydration_entries").select(...)`	SDK GET to `<project>/rest/v1/hydration_entries` with encoded filters and bearer authorization.
`supabase.rpc("commit_health_mutation", args)`	SDK POST to `<project>/rest/v1/rpc/commit_health_mutation`; PostgreSQL runs the function.

Code-visible call	Boundary and operation
`supabase.functions.invoke("estimate-meal", {body})`	SDK POST to `<project>/functions/v1/estimate-meal`; a server handler runs.
Server `fetch("https://api.openai.com/v1/responses", ...)`	A second HTTPS request, authenticated with a server-held provider key.

Read, login and function examples

For a table read, the repository's chained methods become filters in a GET request. This source-shaped teaching example asks for selected owned fluid columns; real queries add their window, ordering and page conditions:

```
const { data, error } = await supabase
  .from("hydration_entries")
  .select("id, volume_ml, counting_policy, alcohol_status")
  .eq("user_id", userId);
```

The response data is an array of projected rows, for example `[{id: "D1", volume_ml: 250, counting_policy: "beverage_volume_v1", alcohol_status: "nonalcoholic"}]`. RLS can restrict the rows returned. An empty array is different from a request error, and neither means "all time fluid intake is zero" without considering the query's date/coverage. The SDK supplies encoded URL parameters and bearer/project headers; the repository interprets the result.

Password login is a different service contract. The app calls `supabase.auth.signInWithPassword({email, password})`. The installed Auth SDK sends POST to the project's `/auth/v1/token?grant_type=password`; signup uses POST `/auth/v1/signup`, and refresh uses the refresh-token grant. These endpoint shapes were checked in the installed SDK, not invented as application routes. The password is the credential being verified, so an existing user access token is not what establishes this login. No password or real token value is reproduced here.

For a successful SDK login, a simplified return projection is `{data: {user: {...}, session: {access_token: "<secret>", refresh_token: "<secret>", ...}}, error: null}`. Failure supplies an error instead of permission to enter. The auth provider then reacts to session events; simply receiving a 200 response from some endpoint is not the navigation gate. Signup may legitimately return no session pending confirmation.

For an Edge invocation, `.functions.invoke("estimate-meal", {body, signal})` serializes the feature body and sends it to the named server function. The server authenticates the bearer and performs additional consent/quota/domain checks. The caller checks SDK errors, may read the error response's safe `message`,

then parses the success body with a runtime schema. The **HTTP contract** determines transport success/failure; the **feature contract** determines whether the body is a usable estimate. Both are required.

The deletion endpoint is intentionally shown later as direct `fetch`, because its continuation token must remain useful after ordinary authentication is revoked. Storage upload/download/signing uses yet another SDK namespace. These differences are why a single diagram labeled "call backend" is not enough to explain the implemented system. [C04, C05, C15, C23]

Follow the Save action

The fluid screen holds amount, unit, name, date and classification. Its handler calls `saveHydration(userId, input)` in `features/hydration/repository.ts`. That function parses the input with `hydrationInputSchema`, derives `occurredAt` with `entryTimestamp`, converts the amount to milliliters, and calls `createRecords`. The lower mutation layer preserves request identity and invokes the SQL function. [C05, C17]

Here is an actual excerpt from `saveHydration`, shortened only by removing surrounding fields:

```
const value = hydrationInputSchema.parse(input);
const occurredAt = entryTimestamp(value.entryDay);
// Within the row builder passed to createRecords:
volume_ml: hydrationAmountToMl(value.amount, value.unit),
counting_policy: "beverage_volume_v1",
occurred_at: occurredAt,
```

Parsing checks the runtime value, not just the TypeScript annotation. The timestamp distinguishes when the drink belongs in history from when the server inserts it. The canonical milliliter value prevents every later total from needing to reinterpret mixed units. The policy string preserves the counting convention used for the record.

For a source-shaped synthetic request, shorten UUIDs to `U1`, `OP1` and `D1`. The actual outer RPC arguments are:

```
{
  "p_user_id": "U1",
  "p_operation_id": "OP1",
  "p_request": {
    "action": "create",
    "table": "hydration_entries",
    "rows": [{
      "id": "D1", "version": 0,
      "values": {
        "fluid_name": "Water", "volume_ml": 250,
        "category_id": "water", "alcohol_status": "nonalcoholic",
        "counting_policy": "beverage_volume_v1",
        "occurred_at": "2026-09-25T16:00:00.000Z"
      }
    }]
  }
}
```

This example teaches the envelope and meaning of the fields; accepted enum strings must be checked against the cited model when constructing a real call. `createRecords` removes client-supplied owner/version/system fields from values and represents a create with expected version zero. The database derives authorization from the session and compares it to the requested user, rather than trusting `p_user_id` as proof of identity.

From response to visible total

An accepted mutation returns an object with `status: "accepted"` and `data`; conflicts return a different recognized status. `sendMutation` rejects unrecognized response shapes instead of presenting them as a successful save. For accepted fluid writes, it publishes `notifyDataChanged(userId, "hydration_entries")`. Summary and History subscribers refresh the matching resources. The screen finishes its saving state and clears only the input state appropriate to a confirmed save.

The daily read in `getTodayHydrationTotals` builds the start of the local day and the next local midnight, sends `.gte("occurred_at", startISO).lt("occurred_at", endISO)`, orders by ID and collects pages. It selects the fields required for `fluidTotals`. That pure function separates volume consumed, counted volume and unresolved classification. The widget receives those computed values and rerenders.

A local day is not always 24 elapsed hours around daylight-saving transitions. Constructing the next local date and converting boundaries to UTC differs from adding a fixed 86,400,000 milliseconds. Stored timestamps represent instants; the view decides which local day those instants belong to.

If validation fails, no RPC should be sent and the field remains editable. If the server definitively rejects a constraint, the pending operation can be removed so corrected data can form a new operation. If the network response is uncertain, the frozen request remains for a safe retry. Disabling Save helps with rapid taps, but this durable request is what survives the process ending. The reliability chapter follows that distinction through a full meal example.

---

05. Building and evolving the database

A database is not just a place to put JavaScript objects. PostgreSQL stores named collections with declared types and rules, executes queries over them and can commit related changes atomically. **SQL**, Structured Query Language, is the language used to define and query those structures. Sustain's `supabase/migrations` directory records 37 ordered changes that construct and evolve the application schema. [C06]

A **schema** is a namespace inside the database. `public.workout_sets` identifies a table in the `public` schema. Supabase also supplies managed areas such as `auth` for identity and `storage` for file metadata. Sustain's migrations refer to those managed structures; they do not recreate the entire Supabase platform from scratch.

A **table** groups records with the same declared structure. A **row** is one record, such as one completed set. A **column** is a named field, such as `reps`. A **data type** defines what kind of value it can store. `uuid` is a standardized identifier type, `text` stores strings, `integer` stores whole numbers, `numeric(10,2)` stores decimal values with defined precision, `timestamptz` stores instants, and `jsonb` stores structured JSON in a database-queryable representation.

Migrations are executable history

The first vitals migration creates profiles and vital samples, enables row-level security and attaches a new-user trigger. Later migrations add training and food tables, provider metadata, photos, recipes, goals and reliability structures. Hardening migrations revoke earlier direct-write grants and replace the supported write boundary. Reading only the first table definition would describe an obsolete permission model.

An **ALTER TABLE** statement changes an existing table without conceptually replacing every user record. A migration can add a column, backfill values, attach a constraint or change function permissions. Applying migrations in order matters: a function referring to `version` cannot work before that column exists. The maintained SQL harness creates simplified platform prerequisites, applies the full order and exercises behavior. It verifies repository compatibility in isolation, not deployment to a particular project.

Supabase's migration tooling tracks applied changes and can apply pending files to a linked database. A reviewed migration should be rehearsed on isolated data and inspected for backfill cost before deployment. The official migration guide supports this workflow; ad hoc dashboard edits make source and hosted history harder to reconcile. [R01]

Keys and relationships protect meaning

A **primary key** uniquely identifies a row. `workout_sessions.id` distinguishes two sessions even if they have identical titles and times. `profiles.id` is both its primary key and a foreign key to `auth.users.id`, so the profile shares the account's identity. Several other one-per-account tables use `user_id` as their primary key.

A **foreign key** requires a reference to identify an existing parent. A set's `session_id` points to a workout. The stronger current relationship pairs `user_id` and `session_id` against the parent's `user_id` and `id`. This prevents a set owned by Alice from referring to Bob's session, even though Bob's session ID exists. A filter in the interface would not enforce this for a malicious or buggy client.

The following connected records are synthetic projections with only teaching fields shown:

Table	Example record
`auth.users`	`id=U1`; account identity managed by Supabase Auth
`profiles`	`id=U1`, `display_name="Alex"`
`workout_sessions`	`id=W1`, `user_id=U1`, `title="Upper body"`, `version=1`
`workout_sets`	`id=S1`, `user_id=U1`, `session_id=W1`, `exercise_name="Row"`, `set_number=1`, `weight=40`, `reps=12`
`workout_sets`	`id=S2`, same owner/parent, same exercise, `set_number=2`, `weight=40`, `reps=11`

Deleting W1 cascades to its sets because they have no independent completed-session meaning. A foreign key with `ON DELETE CASCADE` requests that cleanup inside the database operation. It does not delete arbitrary files from Storage or copies exported to another device.

Core ownership and relationships

auth.users	1 → 1	**profiles / account_setup**
auth.users	1 → many	**vital_samples / nutrition_entries / hydration_entries / cardio_entries**
auth.users → workout_sessions	1 → many	**workout_sets**
vital_samples (weight)	1 → up to 3	**progress_photos → private object path**
user_food_profiles / food_recipes	optional reference	**nutrition_entries + copied snapshot fields**
coach_threads	1 → many	**coach_messages → coach_actions**

The drawing groups related tables for readability. Owner-aware foreign keys protect the important parent/child links; copied meal values remain historical facts even when a reusable source changes.

Constraints express rules independent of a screen

`NOT NULL` requires a value. `DEFAULT now()` supplies insertion time when omitted. A CHECK constraint can require a positive vital value, an allowed kind or a bounded rep count. A UNIQUE constraint prevents duplicate combinations, such as the original session/exercise/set-number identity. These rules still run when the request comes from an old app, a server function or a developer tool.

Defaults are conveniences, not complete validation. Defaulting `source` to manual does not prove the caller should be allowed to create an imported record. Similarly, a UUID with valid syntax is not proof of ownership. The mutation function, active-session check, access policies and constraints solve different parts of the problem.

Ordinary columns suit values the app filters and aggregates frequently: owner, date, kind, volume, calories and version. Nested JSON fits variable structured snapshots such as recipe ingredients, operation requests/responses and calculation metadata. The recipe table checks that ingredients is an array of bounded length, while domain/RPC validation must do more to verify the content. JSON flexibility does not remove the need for a schema; it relocates some rules into code.

Why separate set rows rather than one workout document?

Separate rows let the database search a user's exercise history and filter by muscle group without fetching every workout document. Each row can enforce numeric constraints and participate in foreign keys. Sustain then uses one SQL transaction to keep parent and sets together. The cost is joins, more IDs and a replacement protocol when editing an entire workout.

An alternative would store a session with `sets: [...]` in JSON. Reading or saving the whole workout would be simpler. For a local-only workout notebook, that may be sufficient. To ask for the strongest row performance over 90 days, the backend would need to expand JSON arrays or maintain a separate index/aggregate. Fine-grained validation and concurrent edits would need explicit rules. A document database could similarly group a workout naturally, but cross-document ownership and consistent analytics would not appear automatically.

The selected structure fits a tracker that learns from sets and derives history-based guidance. Recipe ingredients make a different tradeoff because they are saved as a bounded recipe snapshot and commonly processed as one unit. A relational database can deliberately use both columns and JSON rather than making every feature choose the same representation.

---

06. The actual tables, indexes and database programs

This chapter describes the effective repository design after the full migration chain, not every superseded definition. It focuses on columns that explain relationships and calls. The appendix points to the migrations for exact types, complete checks and less-visible compatibility fields. [C06, C07, C21, C22]

Accounts and the health record tables

`profiles` holds display/name fields and current targets such as calories, protein override, weight, BP and fluid values plus calculation metadata. Its key is the Auth user's ID. `account_setup` holds preferred name, unit choices, current onboarding step, completion/dismissal state, a version and updated time. These are separate because progressing through setup should not be inferred from missing health goals. An established account can have no calorie goal without being a new account.

`vital_samples` stores one numeric reading per row: `kind`, `value`, `unit`, `occurred_at`, owner and identity. `correlation_id` pairs BP components. `source`, `external_id` and `source_name` preserve provenance. `deleted_at` is a tombstone; `version` detects stale edits and `change_seq` orders changes for synchronization. Systolic and diastolic are related rows, not a single slash-delimited text value.

`workout_sessions` stores title, muscle groups, times and session context. `workout_sets` stores exercise name/order, set number, left/bilateral load/reps and optional right-side values. `cardio_entries` stores activity type, duration, optional distance, occurrence time and source/deletion metadata. A set's parent session provides transaction and editing boundaries; cardio is independently meaningful as a single completed activity.

`nutrition_entries` stores one consumed food row with a shared `meal_log_id` for grouping. There is no separate meal parent table required for those rows. Fields preserve food/brand, amount/unit, serving count, consumed mass/volume, nutrients, entry method, nutrition source, date and optional reusable-profile/recipe references. `hydration_entries` stores beverage name, canonical volume, classification, counting policy and occurrence time. `saved_drinks` stores private reusable default choices rather than completed consumption.

Reusable information and files

`food_catalog_products` and `food_barcode_lookups` form the provider-backed shared catalog/cache. They are not user meal tables. Products preserve normalized serving bases, nutrients, provider identity, source URL, normalization version and expiry. The barcode lookup row points to a product or remembers a temporary not-found result.

`user_food_profiles` belongs to one account and can reference a catalog product while storing the user's reusable label values. Private corrections should not silently alter the shared provider record. `food_recipes` holds name, description, yield, ingredient JSON snapshots and archive/update fields. Archive hides an item from normal choices while preserving referenced history.

`progress_photos` stores owned metadata, a weight-sample reference, immutable object path, local day/slot and file properties. The binary JPEG lives in the private `progress-photos` bucket. `storage.objects` is the Storage service's object catalog, not a column containing the JPEG in Sustain's application table. A signed URL is a temporary viewing capability derived from an authorized path; it is not the permanent record ID.

AI, streak and reliability tables

`coach_profiles` stores planner-related preferences and retained compatibility fields. `coach_threads`, `coach_messages` and `coach_actions` hold server-side conversation/action records with same-owner relationships. The current UI generates a session plan, so these tables should not be read as a promise of a visible chat feature. `coach_daily_usage` and `meal_estimate_usage` enforce server-side quotas. The prepared `ai_processing_choices` table stores minimal versioned choices, not full prompts or images.

`streak_rules` stores dated configurations; `streak_goal_snapshots` stores target history; `streak_activation` distinguishes initialized tracking; `food_day_completions` is retained storage even though the former completion controls are retired. Derived streak results depend on records and effective dates, rather than a single incrementing counter that would become wrong after a backdated edit.

`mutation_receipts` has composite key `(user_id, operation_id)` and JSON request/response. `deleted_record_ids` reserves hard-deleted identities by owner/table/record. `vital_sync_counters` stores a per-account change counter. `account_setup_receipts` is a separate setup protocol. `biometric_device_credentials` stores a hashed sign-in secret plus device/expiry/revocation context, not a plaintext password. `account_deletion_jobs` stores a hashed continuation capability and deliberately avoids a cascading Auth foreign key so deletion can still be confirmed after Auth is gone.

An index is a maintained route to matching rows

Without a useful index, a query may inspect many unrelated rows to find one user's records. An index stores ordered lookup information alongside the table. It speeds matching/filtering at the cost of disk space and extra work on every relevant write. The order of its columns matters: an index beginning with `user_id` is naturally useful for an account-scoped query.

Actual index or key	Intended relationship to app queries
`vital_samples_user_occurred_at_idx`	Account and descending time support scoped reading history.
`vital_samples_user_correlation_idx`	Partial index for non-null BP correlation lookups.
`vital_samples_user_source_external_kind_idx`	Uniqueness for imported external identity within owner/source/kind.
`vital_change_cursor` on owner and change sequence	Ordered incremental vitals changes and unique per-owner sequence positions.
`workout_sets_user_exercise_created_at_idx`	Account/exercise/time history used by exercise guidance; case-insensitive matching may need different indexing to be efficient.
Nutrition/hydration time indexes	Account-scoped day/history reads; exact usefulness depends on filter/order and query plan.
`food_recipes_user_updated_idx`	Recent active recipes; its predicate excludes archived rows.
Receipt primary keys	Find a prior result by account and operation ID without scanning all requests.

An index existing is not a benchmark. Some current reads page by ID while a time-oriented index exists. PostgreSQL can choose a different index or a scan. Measure real plans and row counts using EXPLAIN on representative nonprivate test data before proposing another index. Blindly indexing every column makes writes and storage more expensive without establishing a benefit.

Functions, triggers, grants and policies

A database **function** is a named program executed inside PostgreSQL. `commit_health_mutation` validates and commits supported health writes. `read_vital_changes` reads a bounded change window. `save_account_setup` combines setup version checks with receipts. Quota, biometric and deletion functions implement their respective protected operations. A function can do multiple SQL statements within one caller transaction.

A **trigger** runs automatically at an event. `create_profile_for_new_user` creates a profile after Auth inserts a user. `create_account_setup` establishes onboarding state. `bump_record_version` assigns version one on insert and increments on update. `assign_vital_cursor` increments the account counter when a vital changes. The caller does not need to remember a second request for these database-owned changes.

A **grant** permits a database role to attempt an operation. **Row-level security**, RLS, narrows which rows that role may access. Sustain's select policies generally compare `auth.uid()` to the owner. Later hardening revokes direct writes for core health tables even where older policies exist; authorized SQL functions are the supported write route. A restrictive deletion policy also blocks access while account deletion is pending. [R02]

`SECURITY DEFINER` functions execute with their owner's privileges. That is useful when ordinary users may not write the underlying tables, but dangerous if a function accepts unchecked identities or constructs unsafe SQL. Sustain narrows accepted tables/fields, fixes search paths and explicitly calls active-session/ownership checks. RLS is not an excuse to skip those checks when elevated execution can bypass ordinary row restrictions.

The review's isolated SQL tests cover cross-account relationships, grants, stale versions and deletion guards. They do not prove every hosted role/configuration matches the repository. Historical records report hosted migrations through `202609220002`; the prepared privacy migration `202609250001` needs its own authorized rollout and verification.

07. Signup, sessions, onboarding and secure entry

Authentication answers who is making a request. Authorization answers whether that identity may perform this operation. An email/password login establishes identity through Supabase Auth; it does not authorize access to every database row. A successful Face ID prompt verifies a person to the device; it does not by itself create a Supabase account or grant arbitrary server access.

Signup and confirmation

The signup screen validates email/password and optional name, then uses `supabase.auth.signUp`. The SDK sends the signup request to the Auth service. The configured confirmation flow can return a user without an authenticated session. The screen must show confirmation instructions rather than treat account creation as permission to enter protected screens. The Auth user insertion triggers creation of the application's profile and setup rows. These operations connect identity to application state without having the client race to create those rows afterward. [C03, C06]

The native callback URL uses the app's configured scheme. The auth provider parses a callback and either calls `exchangeCodeForSession` for a code or `setSession` for returned token material supported by that flow. A deep link is an input to validate, not an automatic instruction to navigate to arbitrary content. Password recovery belongs to the same account; it should not create a replacement account merely because a confirmation screen is involved.

The source contains a prepared branded signup-email template. Historical deployment notes say the live default template remains unchanged because customization was deferred after a provider/plan restriction. That is a concrete example of repository content differing from active service configuration. A local HTML template is not proof an email was sent with that design.

What a session contains

After `supabase.auth.signInWithPassword({email, password})`, the SDK returns session information or an error. A session includes an access token, a refresh token and user data. The access token is presented as `Authorization: Bearer <token>` when calling account services. A **JWT**, JSON Web Token, is a signed token format that can contain claims such as user and session identifiers. Reading a token's text is not verifying its signature; the service performs verification.

A refresh token allows the SDK to obtain a replacement access token while the session remains valid. Refresh is not the same as remembering an email address. In Sustain, `persistSession: false` keeps the Supabase session process-local. The app subscribes to `onAuthStateChange`, updates matching account state and enables/disables refresh around lifecycle events. After the process ends, a remembered identity or

separate Face ID credential can help sign in again; an ordinary persisted refresh-token session is not the selected design. [C04]

`require_active_session` checks the authenticated account and the token's session identifier against the server's active session state. This matters because a cryptographically valid, unexpired token may refer to a session the application has revoked. Sensitive database writes invoke this stronger check. Client account assertions also prevent a delayed response for U1 from being applied after the visible account has become U2.

The provider uses current refs/generations to ignore obsolete asynchronous security reads. For example, a request started for Alex must not mark another account's security ready after Alex signs out. Token refresh within the same account should not unnecessarily invalidate an ongoing security read. These are application lifecycle rules layered around the SDK, not features supplied by React automatically.

Onboarding is durable progress, not a collection of missing fields

`features/onboarding/repository.ts` reads `account_setup` and saves through `save_account_setup`. The row records current step, units and completion. A missing row or read failure is treated as a problem, not evidence that an established account should see a blank questionnaire. The migration explicitly backfills existing accounts as completed.

The setup write includes a version and operation ID. Before submission, its frozen request is saved under an account-specific key. On a later attempt, the repository resumes the saved operation before assuming the cloud has no setup. SQL compares the version, writes the setup change and receipt, and preserves unrelated goals. Completion is explicit even when every optional target was skipped. [C08]

Suppose Alex advances from Name at version 3 to Goals. The request records expected version 3 and the intended changes. If the server commits version 4 but the response disappears, replaying the same operation returns the recorded result. If another device has made a distinct change to version 4 before a new version-3 request arrives, that new request is stale and must reconcile. Replaying a known operation and applying a new stale edit are different cases.

Entry is a coordinator around the authorities

EntryProvider has numbered attempts and phases such as authenticating, resolving and entering. It hides the underlying form while authentication is pending, waits for the matching session/security/setup state and coordinates route replacement behind the cover. Completion occurs when the target route is active and the app is foregrounded and unlocked. It does not wait for every chart or Apple Health import before showing the app.

ReminderNotificationObserver can choose a valid pending logger destination after login. An incomplete account goes through onboarding first. A completed account can go directly to the intended logger without briefly mounting Summary. Attempt IDs prevent an older callback from completing a newer attempt. A

bounded recovery path exposes retry/sign-out rather than leaving an endless animation. Reduced Motion changes presentation, not access rules. [C02, C20]

The privacy boundary deliberately keeps child screens mounted beneath an opaque cover so drafts survive a lock. It disables pointer and accessibility access to the covered content. That design avoids rebuilding the navigation stack on every unlock, but requires careful lifecycle and focus handling. It does not encrypt the contents of ordinary local databases.

Why choose this session policy?

A simpler supported alternative would persist the normal Supabase session using a secure storage adapter and optionally place a local biometric lock over the UI. The SDK could restore the refresh token on startup. That would reduce the custom credential-exchange surface and improve startup continuity, but would deliberately retain reusable session material between processes.

The current design separates remembered identity, short-lived process session, privacy unlock and device credential. It offers explicit revocation and device-bound biometric sign-in at the cost of more failure modes, custom server code and device testing. This is retrospective engineering analysis, not a claim that the owner experimentally compared every alternative. To justify retaining it, test enrollment, rotation, cancellation, revocation, process restart and missing Keychain values on actual devices, not just the successful login path.

---

08. Face ID: unlock, enrollment and credential exchange

There are two different operations behind the Face ID experience. **Privacy unlock** lets an already authenticated session reveal its retained interface after device verification. **Face ID sign-in** obtains a new server session using a protected device credential. Confusing them would lead to the false claim that a biometric prompt alone logs a user into Supabase.

`getFaceIdAvailability` checks iOS, hardware, enrolled biometrics and the supported facial-recognition type using `expo-local-authentication`. `authenticateWithFaceId` calls `authenticateAsync` with device fallback disabled for this flow. It distinguishes success, cancellation, temporary lockout and inability to start. Browser preview reports that this capability is unavailable. [C09]

Enrollment establishes a separate server credential

While signed in, the user supplies the current password to enable Face ID sign-in. `enrollFaceIdLoginCredential` obtains or creates a local device ID and invokes `biometric-auth` with action `enroll`, account ID, password and device context. The Edge handler independently verifies the credential/account and uses service-only database routines to enroll a device credential. The server stores a hash of a random secret, associated with the account and device, rather than storing the account password for future logins.

The returned secret is stored by `saveFaceIdLoginCredential`. The actual SecureStore options include `requireAuthentication: true` and `WHEN_PASSCODE_SET_THIS_DEVICE_ONLY`. The operating system protects access; the public local marker stores identity/credential IDs for finding the item, not the secret itself. The parser binds the secret to its marker and account. Empty and null phone variants are normalized so an email-only account does not fail identity comparison accidentally.

The ordinary native secure-store adapter now fails when protected storage fails instead of silently using ordinary AsyncStorage. That failure preserves the security boundary at the cost of requiring recovery. A browser key-value fallback has different properties and must never be described as equivalent to the iPhone Keychain.

The exchange is deliberately single-use

On Face ID sign-in, `getFaceIdLoginCredential` reads the authenticated Keychain item, which can cause the native prompt. The client invokes `biometric-auth` with `action: "authenticate"`, credential ID, device ID and secret. Server code hashes the presented secret and atomically consumes/rotates the stored value before producing a session response. The response parser checks the account identity and expected token/next-secret fields.

Face ID sign-in sequence

1. **Keychain:** verify the person and release the device secret.
2. **Edge + PostgreSQL:** verify device binding and expiry, consume the old secret and store the next hash.
3. **Auth service:** create the authenticated session returned by the handler.
4. **Client:** persist the next secret before installing the returned session.
5. **Lifecycle gate:** wait for active iOS state, call `supabase.auth.setSession`, then resolve entry.

A database update, Keychain replacement and session installation are separate operations. There is no transaction spanning all three.

Why rotate? If the same device secret remained reusable, a copied secret could mint later sessions until revoked. Rotation narrows reuse: after acceptance, the old value must not work again. But it introduces an uncertainty window. If the server rotates and the reply never reaches the phone, the phone still has the obsolete secret. Unlike a meal receipt replay, this credential flow does not promise transparent recovery of the same session result. Password sign-in and reenrollment are the fallback.

Saving the new Keychain item can also fail after server rotation. The client removes the now-unusable old item before creating the protected replacement, avoiding an additional authenticated update prompt. If replacement fails, there may be no working local credential. That is a supported recovery case, not evidence that rotation and local storage were atomic.

Why wait for the app to become active?

iOS can resolve a biometric/Keychain operation before React Native receives an active lifecycle event. Installing the session while the app is still inactive can cause the normal privacy-lock logic to lock that new session and cancel entry. The actual `waitForBiometricForeground` helper listens for active/background events, rechecks after subscribing to avoid a missed event, and has a ten-second timeout.

On actual backgrounding or timeout, the client reports interruption and retains the rotated credential for a future attempt. It does not bypass the lock to make navigation look successful. Once active, `supabase.auth.setSession` installs returned access/refresh tokens; a failure there is also shown as recoverable. The same-session unlock result is separately bound so a late successful prompt cannot unlock a different account.

Server routines enforce credential expiry, revocation and device/account binding; password/email changes revoke relevant credentials. The current source uses a fixed 90-day credential expiry rather than silently extending forever on every use. Device lists and sign-out-everywhere controls help manage server state, but they cannot remotely erase arbitrary copies already exported by a user.

Synthetic tests exercise parsers, rotation, cancellation, enrollment, revoked sessions and the inactive-to-active race. They provide useful evidence of app decisions with controlled adapters. Real prompt timing, biometric-set changes, Keychain accessibility and device lock states require a compiled iPhone build. A screenshot of a browser lock screen does not test these native guarantees.

09. Reliable writes: one meal, one identity, one receipt

Imagine Alex saves breakfast while entering an elevator. The database inserts the meal, then the network connection drops before the phone receives the response. The phone cannot tell whether the save happened. If it simply inserts again, breakfast appears twice. If it discards the draft, it may hide an unsaved meal. The right problem statement is uncertainty about the outcome of a particular operation.

An **operation ID** names the intended save, independently of the IDs of its rows. **Idempotency** means repeating that same operation does not repeat its effect. Sustain implements it with a durable client request and a server **receipt** containing the accepted request and response. Neither a disabled button nor a basic upsert provides all of that behavior. [C05, C07, C13]

Before Save: the draft is not a completed record

The Food screen maintains a meal draft in memory and account-scoped AsyncStorage. Its entries include amount, serving basis and computed nutrient totals. Restoration uses revision guards so a late saved draft does not overwrite text just entered on screen. Draft writes are serialized, and background handling tries to preserve the latest draft. A saved draft means the form can be reconstructed; it does not mean breakfast exists in cloud history.

Suppose the draft contains oats at 1.5 servings and yogurt at one serving. With synthetic values of 150 calories/5 g protein per oats serving and 100 calories/10 g protein per yogurt serving, the proposed meal totals 325 calories and 17.5 g protein. The original basis and consumed totals are distinct values. This distinction becomes important when history is edited later.

`saveNutritionMeal` validates the entries. It may save/reuse reusable food profiles before the meal itself. Those are separate writes and can succeed even if the final meal later fails. It then calls `createRecords` for `nutrition_entries` using slot `meal:create`, a frozen intent and a builder for all meal rows. The generated `meal_log_id` groups the rows, and each food has its own ID.

Persist before sending

`createMutationRunner` in `mutation-model.ts` turns the logical intent into canonical JSON with predictable object-key ordering. This **fingerprint** is an equality representation, not encryption. The account/slot key looks like `healthapp:pending-write:v2:U1:meal:create`. The pending object contains its ID, fingerprint, frozen request and possibly a completed result.

On the first attempt, the runner generates the operation ID and builds row IDs/timestamps once, then writes the pending object before calling the network sender. On retry, it loads the saved request rather than rebuilding it from today's date or generating fresh food IDs. The repository may prepare data again, but the persisted mutation envelope remains authoritative for that uncertain operation.

This teaching projection shows the important states without claiming to show every serialized field:

Moment	Memory	Durable device state	Server
Editing	Oats + yogurt draft	Saved draft, when persistence succeeds	No new meal
First save	Saving indicator	Draft + OP1 with frozen two-row request	Request in flight
Commit	Still waiting	Same pending OP1	Two rows + OP1 receipt
Reply lost	Error/uncertain feedback	Same draft and OP1	Already accepted
Retry	Waiting again	Same OP1, same request	Receipt found; no second meal
Cleanup	Confirmation/reset	Draft cleared, completed pending record removed	Original meal remains

Server decision order

`sendMutation` invokes `supabase.rpc("commit_health_mutation", {p_user_id, p_operation_id, p_request})`. The SQL function requires an active matching session, obtains an operation-specific advisory lock and checks the receipt. An **advisory lock** is a database coordination lock chosen by the application using a key; it only helps operations that follow the same locking convention.

If a receipt already exists, the function compares its request with the supplied payload. Same ID plus same request returns the stored response. Same ID plus different request is rejected. Otherwise it takes the account row lock, rechecks relevant authorization and validates the action, fields, ownership and expected versions before modifying rows. The accepted response and receipt are inserted in the same transaction as the data.

The following is simplified teaching pseudocode, not the full SQL:

```
require matching active session
lock this account + operation identity
if receipt exists:
    require request equals saved request
    return saved response
lock account's write boundary
validate all requested changes
apply changes
insert receipt(request, accepted response)
commit both together
```

A **transaction** groups database changes so they all commit or none become durable. If the second food row violates a constraint, PostgreSQL rolls back the first row and the receipt too. The user does not get half the intended meal from this batch. This guarantee covers rows within that database transaction. It does not cover earlier reusable-label writes, local draft cleanup or a photo uploaded to Storage.

What the client does with the result

The mutation sender accepts only recognized `accepted` or `conflict` replies. It distinguishes known constraint rejections from uncertain network failures. After acceptance, the runner can retain completed data for draft-backed saves. The Food screen clears the draft, calls `completePendingDraftSave`, then resets its visible form. If the process ends after the server reply but before local cleanup, the completed result prevents another insertion when the retained draft is encountered.

Per-slot execution guards prevent two callers in the same process from running the same slot concurrently. Supported browsers also use Web Locks across tabs. Those locks organize local execution; the server receipt remains necessary for separate devices, timeouts and restarts. The client asserts the account before and after work so a late response cannot be consumed by a different login.

If the user changes the draft while an uncertain request is pending, the runner does not silently replace OP1's payload. It rejects the changed intent until the original outcome is resolved. Otherwise a receipt for breakfast could be mistakenly applied to a newly edited lunch. Profile's pending-change controls provide an explicit place to retry/reconcile uncertain operations.

Why an upsert is narrower

An upsert means insert a row if absent or update the conflicting row if present. Reusing row IDs can prevent duplicate rows, but a blind upsert can overwrite later edits and does not inherently remember the original operation's response. It also does not make several independent HTTP writes one transaction. For a single immutable preference, a simpler upsert may be adequate. For a multi-item meal with lost responses and subsequent edits, a request/receipt protocol gives a clearer contract.

The cost is additional durable client state, receipt storage, more server branches and explicit recovery UI. Receipts must not be deleted casually: forgetting an old accepted operation can make a late retry look new. Any retention policy needs a compatible client retry horizon and preserved identity rules. There is no measured receipt-volume problem in this review, so compaction is a proposal rather than an assumed necessity.

Two devices: a version check is not a lock

Alex's phone and browser both read fluid record D1 at version 7 with volume 250 mL. The phone changes it to 300 mL and sends expected version 7. The server accepts, and a trigger assigns version 8. The browser later tries 200 mL with expected version 7. The server returns a conflict containing the current state rather than silently replacing 300 with 200.

A lock prevents certain operations from executing simultaneously while it is held. It does not know the browser displayed version 7 for ten minutes. A version comparison identifies that stale basis even after the phone's lock has been released. Sustain uses both: locks organize transaction ordering and versions protect the meaning of edits.

Core generic batch writes precheck all expected versions before changing the batch. Goal updates use changed-field baselines instead of treating every unrelated target as one inseparable edit, while keeping coupled numeric/calculation metadata together. Last-write-wins would be simpler, but could quietly erase an intentional update. A full merge editor might reconcile more cases but would be more complex for small numeric forms. The chosen conflict path keeps user input and asks for a deliberate reconciliation.

---

10. Offline vitals, outboxes and synchronization cursors

A draft says, "Here is the form I was working on." An **outbox** says, "Here is a durable change the system still owes the server." Sustain gives vitals an outbox because manual readings and imported samples must survive interrupted synchronization. Most other features retain drafts and uncertain save requests but do not share a complete offline queue. Calling the whole app offline-first would hide that distinction. [C10]

A manual reading before the network

The manual vital component parses a positive weight or BP/pulse input, assigns identity and occurrence time, and calls `queueLocalVitals`. BP components share a correlation ID so 120 systolic and 80 diastolic can be displayed as a pair. A nearby pulse can be associated when available. A reading's measurement time is distinct from the time it is queued or committed.

The local store commits samples and their pending operations together. On native, `sqlite-store.ts` uses an exclusive SQLite transaction, with account cache and outbox represented as JSON state rows and operations serialized in the process. On web, IndexedDB provides a transaction over the account state. A **local transaction** prevents the app from saving a visible local reading without its corresponding pending change if the process stops midway through that storage update.

This abbreviated domain-shaped example is synthetic:

```
{
  "id": "V1", "userId": "U1", "kind": "weight",
  "value": 180, "unit": "lb", "source": "manual",
  "occurredAt": "2026-09-25T11:00:00.000Z",
  "createdAt": "2026-09-25T11:01:00.000Z"
}
```

The local representation uses camelCase; cloud rows use fields such as `user_id` and `occurred_at`. Mapping functions translate between them. Server version and change-sequence values become available through accepted/remote rows. A create is represented with expected version zero in the request; a returned version one means the server has accepted the first version.

Successive edits form dependencies

Suppose V1 is queued offline at 180 lb and then corrected to 181 lb before its first upload is confirmed. A naive queue might submit both as independent creates or send an edit with a version it does not yet know. Sustain's store model retains operations and dependencies. `prepareChange` freezes an operation's request only when its predecessor permits a valid base version.

After the create is accepted, `acceptChange` removes that operation and updates the dependent change to use the accepted version. It must not overwrite the visible 181 lb edit with the returned 180 lb create result. The store therefore distinguishes the remote shadow, pending local changes and the merged local presentation. A response can be correct for an older operation without being the newest user intent.

A BP group is saved locally as a batch, but synchronization sends separate per-sample operations. Systolic may be accepted before diastolic if connectivity fails between them. The UI's pairing logic and subsequent retries handle that state; the guide must not extend the local batch transaction into a single cloud BP transaction that does not exist.

Push pending operations, then pull changes

`syncVitals` delegates to `performSync` and prevents overlapping account runs from racing. It iterates queued candidates, calls `prepareChange`, sends the frozen operation through `sendMutation`, then calls `acceptChange` or `failChange`. An uncertain exception retains the operation and stops that push loop. A conflict is recorded with remote state and surfaced for review instead of silently discarded.

It next reads `syncCursor` and calls the actual RPC:

```
const { data, error } = await supabase.rpc("read_vital_changes", {
  p_user_id: userId,
  p_after: cursor,
  p_through: through ?? null,
  p_limit: 200,
});
```

`p_after` means the last consumed change position. `p_through` initially asks the server to choose a ceiling, then preserves that ceiling across pages. `p_limit` bounds each page. The response contains rows, a new cursor, a through value and a done flag. The client validates those fields and rejects a nonfinal page that does not advance, avoiding an infinite loop on an unexpected response.

`applyRemotePage` merges rows and advances the cursor in the same local transaction. If the cursor advanced first and the process ended before storing rows, those rows might never be requested again. If rows are applied without advancing and fetched again, stable IDs and merge rules can tolerate repetition. Atomic local application makes the intended boundary explicit.

Why a sequence is better than an updated timestamp

Two changes can have equal timestamps, clocks have finite precision and client clocks can be wrong. A cursor should identify ordering in the system that accepts changes. Sustain's `assign_vital_cursor` updates a per-user counter row; that update locks the counter until commit. The result is commit-ordered allocation for that account.

A global database sequence alone would not provide the same property. Transaction A might allocate 100 and pause, while B allocates 101 and commits. A reader that consumes 101 and assumes all lower positions are committed could miss A when it later commits. Serializing allocation through the account counter avoids that ordering hole at a cost: concurrent writes for one account wait for one another.

For a synthetic example, a client at cursor 40 requests changes and receives a ceiling of 45. It can page through that bounded range while another device creates change 46. That later change belongs to the next sync. However, the current table is mutable, not an immutable event log. If a row moves from sequence 43 to 46 before its page is read, the earlier row version is not preserved as a snapshot. It will arrive in a later sync. The ceiling bounds work, not a frozen copy of every historical row value.

Deletions and conflict choices

A **tombstone** is a retained record marking that a value was deleted. Vitals retain `deleted_at` so another device can learn that V1 should disappear. If the server simply removed V1 and sent only surviving rows, an offline phone would have no evidence to remove its old cache. Hard-deleted record types instead use reserved deleted IDs in the mutation protocol to stop old creates resurrecting removed identities.

Conflict resolution must use the latest remote version the user reviewed. Choosing a local value based on stale remote state should not overwrite yet another device's intervening change. Recovering a deleted reading creates a new manual identity rather than reactivating the old ID. This preserves the distinction between undoing a local display choice and making a new server record.

The Profile sync view distinguishes pending work, failures and completion. `lastSyncedAt` alone is not a promise that every operation succeeded: the result can also contain pending counts and an issue. Account assertions surround network and storage steps; a rejected session triggers account-aware local sign-out without signing out an unrelated newer session.

What this costs and how to test it

The current JSON-state storage is easy to reason about and migrate, but parsing/writing a whole account grows with its history and outbox. A normalized SQLite table per sample/operation could reduce that work, at the cost of more schema migrations and equivalent transactional browser behavior. A server event log could provide replayable immutable history, at the cost of retaining and compacting many more events.

Tests cover interrupted operations, dependent edits, stale responses, conflicts, remote-page merges and tombstones with synthetic data. To extend confidence, exercise actual process termination around local commits, concurrent browser tabs, large history, full device storage and real multi-connection server commits. Those are targeted improvements to evidence, not reasons to claim the current queue has already been proven at arbitrary scale.

---

11. Apple Health, from native sample to displayed reading

Apple Health integration begins with data on the iPhone, not an HTTP request to an Apple health-record website. HealthKit is Apple's native framework for authorized access to health samples. Sustain's JavaScript cannot call an arbitrary Swift API directly, so `@kingstinct/react-native-healthkit` provides the native bridge. The iOS reader checks for NitroModules through `TurboModuleRegistry` and lazily imports the library. A missing compiled module produces a useful rebuild message rather than pretending the browser fallback can query HealthKit. [C11]

Build configuration and availability

`apps/mobile/app.json` includes the HealthKit config plugin. It sets a read-purpose description explaining weight, BP and paired pulse and that readings sync to the private Sustain account. It sets `NSHealthUpdateUsageDescription: false` and `background: false`. The plugin participates in native capability/entitlement configuration; the installed binary must actually include it. There is no configured HealthKit background-delivery pipeline in this source.

`healthKitAvailability` checks the compiled bridge, loads the module and calls `isHealthDataAvailableAsync`. These checks answer whether the integration can be attempted on the current device/build. They do not prove the user has granted every read type, that a sample exists or that the cloud is reachable. Expo Go cannot acquire a missing native library by receiving a new JavaScript bundle. A development build with that library is necessary. [R03]

Requesting access without claiming too much

The actual `requestHealthKitAuthorization` calls:

```
await healthKit.requestAuthorization({
  toRead: [...healthKitReadAuthorizationTypes],
});
```

The array contains body mass, blood-pressure systolic, blood-pressure diastolic and heart rate quantity identifiers. The BP correlation identifier is used for querying, but not included in this read-authorization array. Repository comments document a native exception concern with that combination, and tests preserve the intended set. This is a concrete implementation choice, not a generic instruction to authorize every HealthKit type.

Apple deliberately does not reveal whether a user denied a particular read permission: a query can look empty when access is denied. Successful authorization-dialog completion therefore cannot establish that every type was granted. Sustain's `connected: true` records its integration choice after the authorization call, not an independently verified all-types permission status. An empty import must not be shown as proof of zero readings or definite denial. [R04]

`connectHealthKit(userId)` checks availability, requests authorization, loads account-scoped integration state and persists the connected choice. The user-facing settings workflow can then initiate import. Foreground/focus synchronization and the Profile sync action coordinate further work through the shared health-sync path; this is not continuous background observation while the app never runs.

The actual queries

`readHealthKitData(anchors, options)` sets a start date one calendar year before now. It calls `queryQuantitySamplesWithAnchor` for body mass with the previous weight anchor, date filter, `limit: 0` and requested unit `lb`. It calls `queryCorrelationSamplesWithAnchor` for blood pressure with the BP anchor and the same date filter. During initial pulse backfill it deliberately omits that BP anchor so prior BP correlations can be reconsidered for nearby pulse.

An **anchor** is an opaque position supplied by HealthKit for continuing a query. Sustain stores it, passes it back and receives a replacement; it does not parse it as a timestamp or use it as the Supabase cursor. HealthKit anchors and cloud change cursors track changes in different systems.

The native bridge returns weight samples/new anchor/deleted samples and BP correlations/new anchor/deleted samples. A source-shaped synthetic weight sample, before app mapping, is:

```
{
  uuid: "<HealthKit sample UUID>",
  quantityType: "HKQuantityTypeIdentifierBodyMass",
  quantity: 180.25,
  unit: "lb",
  startDate: new Date("2026-09-25T11:00:00Z"),
  sourceRevision: { source: { name: "Example scale" } }
}
```

Only relevant fields are shown; the bridge supplies a richer native object. Sustain filters nonfinite/nonpositive weights, rounds to two decimals, converts `startDate` to an ISO timestamp and retains a usable source name. It hides unhelpful proxy source names rather than presenting internal bridge labels as device provenance.

For BP, each correlation contains quantity objects. The reader finds both systolic and diastolic types and skips incomplete pairs. Both rows use the correlation UUID as external identity, distinguished by kind, and share a derived correlation ID. `pressureInMmHg` converts kPa by multiplying by 7.50062 and inHg by 25.4; otherwise it returns the quantity value. The expected unit path should be verified with native samples, because this fallback is not a general unit-conversion validator.

For each complete BP correlation, `pulseNearBloodPressure` calls `queryQuantitySamples` for heart rate in the two minutes before and after the BP timestamp, requests `count/min`, ascending order and at most 12 samples. It filters positive finite values, prefers the same source name and chooses the closest timestamp. The pulse value is rounded to an integer and stored as bpm. A set prevents importing the same pulse UUID twice within the batch. This is a proximity heuristic, not proof that two devices measured the same physiological event.

Trace the synthetic weight all the way through

`importHealthKitData(U1)` first calls `syncVitals(U1)`. If that cloud sync fails, the import stops before querying HealthKit. It then loads saved anchors and calls the native reader. This order helps reconcile server deletions/known records first, but means the current import is not fully offline even though HealthKit itself is local.

For the example sample, `healthKitRecordId(userId, kind, externalId)` uses UUID v5 with the account ID as namespace and `kind:externalId` as its input. The same account/kind/external sample produces the same internal ID on repeated processing. Another account gets a different internal identity. A deterministic ID is stable naming, not proof of authorization.

The mapper constructs a `VitalSample` with `userId`, kind weight, value 180.25, unit lb, occurrence time, source `healthkit`, external ID and source name. It compares all cached IDs, including tombstones, and filters already-known IDs. Retaining tombstone IDs prevents a manual hide/deletion from being undone by reimporting the same external sample.

The mapper calls `queueLocalVitals(samples)`, entering the same durable cache/outbox used by manual vitals. `syncVitals` sends them through the mutation RPC and pulls current remote changes. Only after a successful concluding sync does `saveHealthKitSyncState` store the new native anchors, pulse-backfill flag and last-import time. Counts returned to settings reflect newly queued weight and BP groups, not every sample HealthKit examined.

One sample crosses two independent storage systems

On-device HealthKit	→	Local processing/store	→	Network / PostgreSQL
UUID + 180.25 lb + Date + source		Stable owned ID, mapped VitalSample, cache + outbox transaction		Receipted write, version, change sequence

Summary/History load the local vitals representation after synchronization. Display conversion or bucket averaging happens later; it does not rewrite the imported raw value.

Summary's vitals resource and History read the cached model, filter deleted readings, group BP and prepare charts/text. Display can be converted according to units, while provenance remains attached. New values become visible through cache reload/health-sync refresh paths rather than a direct HealthKit object being rendered as a chart point.

Failure, repetition, edits and deletion

If the final cloud sync fails after local queuing, imported readings remain queued and the old anchors remain. The next import can see samples again, but deterministic IDs and cached-ID filtering avoid ordinary duplicates. Anchors therefore advance after the downstream handoff, not before it. If the native read throws partway through its queries, the batch is not returned; the old state permits retry.

The reader returns deleted weight UUIDs and deleted BP correlation UUIDs. The importer maps those to owned internal IDs, finds live imported matches and queues tombstones. Deleting a BP correlation tombstones its systolic and diastolic rows. Associated pulse uses its own external ID and there is no standalone anchored heart-rate deletion stream here, so arbitrary pulse deletions are not comprehensively mirrored. Existing cached IDs are filtered rather than blindly overwritten; edits represented by HealthKit as deletion/new samples follow that identity behavior.

No steps, sleep or workouts are imported by this active pipeline. Helper mappings for workout types do not make them active queries. Browser preview tests mapping, UI and synthetic synchronization, but cannot exercise the native privacy dialog, real anchors, permissions changing in Settings or locked-device accessibility.

An alternative would query a full recent window on every refresh and deduplicate everything. That is easier to understand initially, but does repeated work and needs a separate deletion strategy. A broader native observer/background pipeline could improve timeliness but would introduce background delivery, resource limits and more lifecycle testing. For personal foreground tracking, anchors plus explicit refresh are a reasonable intermediate design, provided finite lookback and pulse limitations remain visible.

12. Lifting, cardio and exercise guidance

The workout interface captures a structured session, not merely a note saying exercise happened. A muscle-group selection constrains exercise rows; each exercise has ordered sets with loads and reps. Bilateral and single-side modes represent different inputs. A blank load is incomplete, while an explicit zero is valid for bodyweight work. Preserving this distinction in form state prevents the common error of treating an empty string as numeric zero. [C12]

From form to one transactional workout

The Lifting screen maintains an account-scoped draft, including exercise order, set fields and planner notes when present. Saved-draft restoration must not race newer editing. The save handler turns completed input into `WorkoutSetInput` values and calls `saveWorkout`. The repository parses every set with `workoutSetSchema`, requires at least one set and checks that every exercise's muscle group is selected.

`saveWorkout` calls `runMutation` with slot `workout:create`. Its frozen request uses `action: "create_workout"`, a session ID, title, muscle groups, occurrence time and the entire set array. It generates set IDs and set numbers per exercise. The current request stores loads in lb and includes `side_mode`, `right_weight` and `right_reps` where needed. These are actual field names; the synthetic values below are a projection, not a complete request.

```
{
  "action": "create_workout", "id": "W1",
  "title": "Upper body", "muscle_groups": ["back"],
  "sets": [{
    "id": "S1", "exercise_name": "Row", "exercise_order": 0,
    "muscle_group": "back", "set_number": 1,
    "weight": 40, "weight_unit": "lb", "reps": 12,
    "side_mode": "bilateral", "right_weight": null, "right_reps": null
  }]
}
```

The SQL function validates the shape, allowed groups and numeric bounds, inserts the session and then its sets, and records the accepted receipt. A bad set rolls back the parent and other sets. The cloud does not expose an intermediate empty session created by a separate client request. Completed response retention protects draft cleanup just as for meals.

`replaceWorkout` takes the session ID and expected version, validates a replacement set array and sends `action: "replace_workout"`. The server replaces the child set collection within the transaction and increments the session version. Two devices cannot silently replace each other's versions. Set IDs may change on replacement, so they are not immutable identities for an exercise across all future session edits.

Read paths and a refresh limitation

Training history pages owned sessions and owned sets, then associates sets with their parent. Ordering by occurrence time is a presentation decision after collecting the relevant records. Separate resource loads prevent a food-read error from blanking exercise history. Focus and foreground refresh re-read the appropriate resources.

The source review found a concrete invalidation mismatch: workout create/replace requests do not include `table`, while the generic sender emits `String(request.table)` for non-goal writes. That yields `"undefined"`, but Summary and History subscribers look for `workout_sessions`. Focus reload can refresh the display, but this event does not immediately invalidate the expected category. This is a source-confirmed gap, not evidence of failed transaction storage. A proposed fix would map workout actions to their affected category and verify a retained view refreshes without navigation. [C05, C12, C18]

The 90-day guidance heuristic

`getExerciseGuidance` queries the user's recent sets with an exercise-name match, a 90-day creation-time boundary and a bounded result count. It chooses strongest load, then total reps and recency according to its domain rules. If at least two working sets meet the rep threshold, it can suggest increasing load: the implemented increment is 2.5 at loads up to 50 lb and 5 above that threshold.

For a synthetic example, two 40 lb sets of 12 and 12 reps can meet the increase condition; 12 and 10 do not. The rule is a product heuristic. It is not an individualized physiological model and does not establish that the next load is safe for a particular person. It also reads set creation time, which differs from a backdated session occurrence time. Editing/replacing sets can affect that history basis.

An alternative would preserve a formal exercise catalog, stable exercise IDs and periodized progression models. That could improve grouping across renamed exercises and provide more detailed programming, but it would require catalog maintenance and clearer model assumptions. The current private-history suggestions avoid seeding exercises the user has never logged, while name matching remains less exact than a stable catalog identity.

Cardio is a separate record and draft

Cardio captures activity type, positive duration, optional nonnegative distance, notes and entry date. Its save path validates the domain input and uses a receipted `cardio_entries` create. A synchronous guard blocks repeated submission while pending. Draft storage lets the user visit History and return without accidentally saving or clearing unfinished input.

Cardio deletion is a versioned change with soft-deletion semantics in the hardened write path. A manual record remains distinguishable from imported source metadata even when the interface hides redundant attribution. Lifting and cardio appear together in Exercise history but retain different schemas and save boundaries.

Tests cover zero versus blank loads, side modes, exercise ordering, guidance, history edits, lost-response saves and cardio navigation. Synthetic browser cases exercise completed lifting/cardio flows. Actual keyboard behavior and native touch/drag interactions still need device validation. To scale long histories, first measure query count, returned set volume and client grouping cost; do not assume a new database is needed simply because the number of sets increases.

13. Food labels, meal snapshots and reusable recipes

Food tracking separates three questions: what a food label says, how much the person consumed and what was recorded at that time. If those concepts share one mutable object, fixing a label today can silently rewrite last month's calories. Sustain uses reusable bases for future selection and immutable consumed-value snapshots in completed history. [C13]

Serving arithmetic before storage

A food basis can describe nutrients per serving, mass per serving, volume per serving or a household quantity. Suppose an oats label says one serving is 40 g, 150 calories and 5 g protein. Entering 60 g gives `60 / 40 = 1.5` servings, so the consumed snapshot is 225 calories and 7.5 g protein. The original 40 g basis and the consumed 60 g are different fields.

If a milk label says one serving is 240 mL and 120 calories, 120 mL is half a serving and 60 calories. A tablespoon-to-grams conversion cannot be inferred just because both numbers describe a food. It requires a known household or mass/volume basis. The model makes conversions available only when the necessary basis exists; it does not invent density.

Optional nutrients remain unknown when absent. If one recipe ingredient lacks sodium, reporting a known total sodium as zero would be false precision. Known zero means an explicit measured/entered zero; unknown means insufficient data. Runtime schemas enforce plausible shapes and bounds, but cannot prove the user copied the package correctly.

Add food and maintain a draft

The Food route coordinates a meal type, date and draft entries. Food editor modules handle label creation, serving selection and amount changes. `getFoodSuggestions` reads active owned profiles, recent nutrition entries and recipes, then ranks matches. It is a bounded private search: current queries limit profiles and recent records and deduplicate suggestions; it is not a complete text-search index over every food in the world.

A selected suggestion supplies a basis, not a completed history row. The amount editor computes a draft entry with serving count, consumed quantities and totals. The user can add several items, correct them and discard or save the meal. Async draft restoration is guarded against newer local changes, and clearing a draft also resets mounted forms so an old retained component cannot immediately save it back.

`saveFoodProfile` resolves reusable identity through private ownership and relevant catalog/barcode/content rules. A corrected private barcode label can be reused ahead of provider data. Archiving a profile uses a versioned mutation to set `archived_at`, preserving old references. A historical nutrition row already contains the nutrient values it needs even if the reusable profile changes.

What gets frozen in a meal

`saveNutritionMeal` validates entries, saves/reuses profiles when requested and builds one `nutrition_entries` row per item. Rows share `meal_log_id`; store optional `food_profile_id`/`recipe_id`; copy name, brand, barcode, serving/household information; preserve consumed quantity/unit/count, all nutrient totals, source/method/note and occurrence time. All meal rows use the same receipted batch.

The distinction between `source` and `nutrition_source` matters. The former retains broad database-origin compatibility such as manual/barcode. The latter identifies whether nutrient knowledge came from a manual label, Open Food Facts or AI. An AI estimate saved through the ordinary logger should not lose the fact that its nutritional basis was estimated.

Consider a completed oats entry with 1.5 servings and 225 calories. Later the label is corrected to 160 calories per serving. Opening that old entry still shows 225; it does not join the current profile and recalculate to 240. Editing only its consumed amount to two servings uses the saved ratio `225 / 1.5 = 150`, giving 300 calories. Applying a newly corrected label would be a distinct product operation, not an automatic historical rewrite.

`getFoodHistory` collects paginated owned rows, maps database fields to typed history entries and sorts by occurrence time. Daily/monthly totals derive from those saved values. `deleteFood` and amount edits require the current version. A food item can be deleted within a meal grouping without treating the meal as a separate required parent object. Related views refresh through the nutrition data-change category.

Recipe construction and reuse

`saveFoodRecipe` stores a recipe name, optional description, positive yield and ingredient snapshots. Ingredient amounts are resolved using the same food model. The recipe calculation sums ingredient totals and divides by yield. Suppose a batch totals 1,200 calories and 60 g protein and yields four servings: one serving is 300 calories/15 g protein, and half the entire batch is 600/30.

The recipe basis represents a whole-recipe household amount using the yield relationship, so users can log fractional batches as well as servings. An ingredient snapshot protects the recipe from later food-label edits. Logging the recipe then creates a consumed snapshot in nutrition history, providing another deliberate historical boundary.

The current workflow does not expose nested recipes as an unrestricted recursive composition system. Avoiding recursive dependencies simplifies calculation, unknown-nutrient propagation and cycle prevention. It also means editing a reusable ingredient is not automatically editing every recipe that once used it. Recipe archival removes future choices without rewriting completed meals.

Why not store only references?

References would use less duplicated data and make label corrections propagate automatically. That is desirable for a live catalog display. It is less desirable for a historical diary where the recorded assumptions should remain explainable. Snapshots duplicate names, serving fields and nutrient values, but make exports and old entries understandable without retrieving the exact old label version.

A credible alternative is a versioned food catalog: each label edit creates a new immutable label version, and meals reference that version. It reduces repeated snapshot fields when many meals share one version, but adds version creation, retention, joins and import/export requirements. For a personal tracker, copying the bounded fields into each consumed row is a reasonable balance. A large shared catalog with formal provenance might make versioned references more attractive.

Tests cover servings, mass/volume/household conversions, unknown optional nutrients, recipes, canonical barcode identity, draft races and history arithmetic. They verify arithmetic against input values, not the nutritional truth of a provider label or AI estimate. The UI's editable review remains part of the data-quality boundary.

---

14. Barcode lookup and external product data

A barcode is an identifier, not an encoded nutrition label. Scanning extracts digits; Sustain must decide whether those digits match a private label or a provider product and then interpret the provider's serving information. The camera, private database and external service perform separate jobs. [C14]

The scanner uses Expo camera support enabled by the app configuration. Camera permission is requested for user-initiated scanning; denial must leave manual entry available. The barcode-normalization logic handles supported UPC/EAN forms, validates digits/check information and resolves canonical/provider representations. Two formats representing the same product should not create two reusable private labels merely because one includes a leading zero.

The repository checks an owned reusable basis where appropriate. For external resolution it invokes `resolve-food-barcode` with the barcode and scanner type. The Edge Function accepts POST, checks a bearer token using the user client and `auth.getUser`, parses the body and normalizes again on

the server. Client validation improves feedback; server validation remains necessary because anyone can construct a request without using the scanner.

Cache decisions and the provider request

The function uses a privileged server client to read the shared `food_barcode_lookups` row. A fresh not-found row returns a meaningful miss without calling the provider again. A fresh product with a supported normalization version returns the normalized cached product. A normalization-version field matters because the same stored provider data may require different interpretation after parsing rules change.

If refresh is needed, the actual server call is a GET to:

```
https://world.openfoodfacts.org/api/v3.6/product/
  <providerBarcode>.json?fields=<selected field list>
```

The line break is for readability. Server code sends `Accept: application/json`, a configured `User-Agent`, and an eight-second abort timeout. It requests product/name/brand, serving/package quantities and nutrition fields. The API does not receive the user's meal history; the barcode is the lookup input. Server identification configuration remains separate from private Supabase service credentials.

A 404 or a recognized product-not-found payload creates a negative cache entry lasting 24 hours. A recognized product is normalized into serving, household, container and nutrient fields, then upserted by provider/provider ID. Its lookup is refreshed with a 30-day lifetime. The response includes `catalogProductId`, name/brand/barcode, source URL, serving bases, nutrients, `complete` and cache status.

The following is a synthetic projection of the response, not a claim that a particular barcode has these values:

```
{
  "name": "Example oats", "source": "open_food_facts",
  "servingWeightGrams": 40,
  "nutrientsPerServing": { "calories": 150, "proteinGrams": 5 },
  "complete": true, "cache": "refreshed"
}
```

Why normalize instead of forwarding raw JSON?

Provider fields can describe per-100 g values, a declared serving, package quantity or incomplete data. Sustain's normalizer produces the food basis expected by the editor. It checks finite positive serving measurements and determines whether the basis is complete enough for its required values. The caller can still review/correct an incomplete or implausible label instead of treating a successful HTTP response as proof of usable nutrition.

On provider failure, a compatible stale product can be returned with `cache: "stale"`. Without such a product, the function returns a provider-unavailable error and the UI permits manual creation. Negative caching avoids repeatedly paying latency for the same miss; stale fallback favors continuity. Both introduce freshness tradeoffs. A new product might remain undiscovered until a miss expires, and a stale label might reflect an older package.

Calling Open Food Facts directly from every client would remove a server hop, but would duplicate normalization/cache behavior across installed versions and make identifying/rate-limiting requests less controllable. The shared resolver centralizes interpretation and cache policy. It does not make the external database authoritative or remove the need to follow its service terms. Current provider payload normalization is covered by fixtures; no live barcode lookup was performed for this report.

15. AI meal estimates and workout plans

AI assistance is useful when input is incomplete or a proposal needs to be generated, but model output is not a trusted database command. Sustain places authenticated server handlers between the client and OpenAI, validates inputs/outputs and sends results back into editable local workflows. The most important boundary is that generating a suggestion is separate from logging a completed health event. [C15, C16]

Meal estimation: exact call path

The AI meal editor gathers text and optionally a selected photo, along with current per-request permission. `estimateMeal(input, signal)` in `ai-meal-client.ts` parses `mealEstimateRequestSchema`, calls `supabase.functions.invoke("estimate-meal", {body, signal})`, checks cancellation and parses `mealEstimateSchema` on success. A successful HTTP response with an unexpected body still fails the client schema check.

Image preparation uses Expo image processing and the app's metadata-stripping helper before transmission. The selected image is resized/re-encoded within bounded dimensions/size; temporary copies are handled separately from the user's original library asset. Requesting a photo is user initiated, and cancellation should not create an empty estimate or completed meal.

The Edge handler validates method/authentication, request size, shape and the current consent version (`2026-09-25.1` in this source). It records a minimal processing-choice receipt before reserving its daily usage quota. These are server checks; hiding the button when permission is missing would not protect a direct caller. Prepared migration/function deployment must precede claims that the live service enforces this newest contract.

Server code calls OpenAI's Responses endpoint using a server-held API key, structured output schema, bounded input/output and timeout. Conceptually, its request contains a model choice, instructions, user description and optional image content, output-format requirements and `store: false`. This projection deliberately omits the full production prompt/schema and is teaching pseudocode:

```
POST https://api.openai.com/v1/responses
Authorization: Bearer <server provider key>
Content-Type: application/json

{ model, input: [bounded text and optional processed image],
  text: { format: <JSON schema contract> }, store: false, ... }
```

OpenAI Structured Outputs constrains the format of generated output; it does not establish that a calorie estimate is true. The provider can also refuse or produce an incomplete response, so the server handles those outcomes and applies runtime/domain validation. `store: false` is not a promise that every provider retention mechanism is disabled; application state and provider data controls are separate matters. [R05, R06]

From output to user-controlled food

The handler parses and normalizes food estimates and returns them for review. The editor shows food names, amounts, nutrients and assumptions, including cooking fats where relevant. The user can adjust, omit or decline them. Accepted items become ordinary meal draft entries with AI provenance. Only Save meal uses the completed-nutrition write path.

Suppose an image could be rice with oil or plain rice. An apparently exact number from a model would hide that ambiguity. Showing the assumption and permitting correction is a product safeguard; a JSON schema cannot infer the actual amount of oil. Nonfood/unidentifiable results should remain an explanation or empty review instead of hallucinated labels being silently saved.

AbortSignal can stop the client waiting and suppress application of a late result. It does not prove the provider never began computation, the server rolled back a quota reservation or no bill occurred. The meal handler's 20-per-day quota can remain consumed on a failed attempt. Automatic paid retries are intentionally avoided. A user-requested new estimate is a new provider operation, not the same receipted health-record mutation.

Workout planning: bounded context, then local drafts

The current planner asks for one lifting, cardio or mixed session, available time, equipment and validated goal/style choices. It can include bounded recent training history only with the current request's choice. The visible workflow no longer exposes general health chat or existing thread reuse. Retained schema fields and legacy repository methods must not be mistaken for current product scope.

`sendCoachMessage(input, signal)` invokes `coach-chat` and validates `coachResponseSchema`. The server verifies authentication and consent, builds compact context and enforces quotas. Current generation uses the deep tier, with source configuration for provider models rather than a guarantee about live deployed secrets. Source limits include three deep or thirty standard requests per user/day and bounded output/tool work; current UI generation does not offer every retained standard-chat path.

The context builder queries training through the authenticated user client. It summarizes recent sets/sessions/cardio and relevant group counts rather than sending the entire account database. When history is disabled, it should not read history merely to discard it later. Tools are bounded read operations with identity supplied by the server, not model-selected arbitrary account IDs or SQL.

The response proposes a workout action. Shared validation checks section completeness, permitted action types, time feasibility, exercise details and effort/rest/technique constraints. Unsafe or unsupported content can be suppressed or returned as caution. A valid shape is necessary but still cannot guarantee the plan suits a person's medical or recovery condition.

`applyCoachWorkout` and draft helpers populate local lifting/cardio drafts. Existing affected drafts require deliberate replacement. Applying a mixed plan involves sequential local writes, not a transaction across both draft stores. Completed history remains unchanged until the tracker Save action. Marking an action applied on the server is separate bookkeeping and can fail without undoing a successfully populated local draft.

Provider failure, persistence and cost

The coach handler can use `save_coach_turn` to persist related messages/actions atomically after a valid result. That database transaction cannot undo the provider request that generated the result. If saving fails, the app may have paid for computation without a durable conversation turn. The coach path refunds application quota reservations on certain provider/save failures; refunding that counter does not refund provider charges.

A lost response after successful model work does not have the same durable operation/result retrieval protocol as a saved meal. A future improvement could assign a generation ID, store bounded job/result state and let the client retrieve the completed result. That would require retention rules, account access controls, cancellation semantics and a distinction between queued/running/failed/completed. It is not justified merely by naming a queue; it is justified if measured duplicate requests or expensive lost results become material.

Tests cover consent failure, context minimization, schema/action validation, quota failure, refusal/incompletion and safe diagnostics with synthetic providers. They do not measure live output quality, real request latency, funded model access or provider retention compliance. Meaningful evaluation would use synthetic meals/planning constraints with expected behaviors, inspect hallucinations and unsafe recommendations, and record results separately from unit-test success.

Why not call the model directly from the phone?

A provider key in the app could be extracted and used outside its quota/UI controls. A server handler protects that credential and provides a common validation/rate boundary for all installed versions. The additional hop adds latency and operational responsibility. A purely local model would avoid sending content to a remote provider, but would introduce device compute, model distribution, quality and update constraints. A deterministic recipe/plan template would be cheaper and more predictable but less responsive to varied descriptions. These are alternatives with different product capabilities, not automatic upgrades.

---

16. Fluids, goals and calculations that preserve their basis

A goal is a user-applied target. A calculator result is a proposal based on particular inputs and assumptions. Keeping them separate lets the user inspect several scenarios without silently changing the app's progress denominator. Sustain's Profile goal forms and onboarding reuse the same helper logic while preserving independent drafts, saved calculations and applied goals. [C17]

Canonical units and fluid contribution

The fluid model accepts mL, fluid ounces and cups, then converts to mL using defined constants. Eight US fluid ounces becomes `8 × 29.5735295625 = 236.5882365`, rounded to 236.59 mL for the stored amount. A cup uses 236.5882365 mL. Displaying a familiar unit does not change the canonical quantity used for summing.

The shared `fluidContribution/fluidTotals` module distinguishes total volume from the app's counted contribution. Under `beverage_volume_v1`, nonalcoholic volume counts fully, alcoholic volume counts zero and unknown classification remains unresolved. Legacy records use their preserved policy. For 250 mL water plus a 330 mL alcoholic drink and 200 mL unknown beverage, total consumed volume is 780 mL, counted contribution is 250 mL and 200 mL remains pending classification.

These are transparent product rules, not a model of absorbed water or dehydration. Classifying a drink changes its counted contribution without changing the physical volume logged. Saving a favorite uses `saveDrinkPreset`, which looks up an owned name and performs a versioned update or create. Saving the favorite and logging consumption are separate operations; a reusable default can exist without a completed drink.

Calorie calculation as implemented

`calculateBmr` uses Mifflin-St Jeor: `10 × kg + 6.25 × cm - 5 × age + category constant`, where the constant is 5 or -161 for the supported equation categories. It validates the supported age/height/weight ranges. `calculateMaintenance` multiplies by a selected activity factor: 1.2, 1.375, 1.465, 1.55, 1.725 or 1.9. This chapter explains the current code, not a medical recommendation.

For a synthetic adult using the male equation at 80 kg, 180 cm and age 30, the resting estimate is `800 + 1125 - 150 + 5 = 1780`. With factor 1.55, maintenance is 2,759 calories/day. A one-pound-per-week planning scenario subtracts 500 to produce 2,259. Display rounding happens after calculation. The loss-rate function permits its defined 0.1-2 lb/week range, while generated targets also pass the app's 1,000-20,000 calorie bounds and other form-level availability rules.

The subtraction is explicitly a short-term planning heuristic, not a dynamic weight-change simulator. Multiplying a fixed deficit by time ignores adaptation and individual uncertainty. A beginner should be able to explain the arithmetic without presenting the target as a guaranteed outcome or the code's lower bound as a clinical safety certification.

Gain scenarios multiply maintenance by the selected 5% or 10% increase. A timeframe comparison works backward from desired change/time to an implied rate, then shows availability under the same supported limits. Viewing comparisons does not apply one. Existing saved maintenance snapshots can support comparisons even after a manual goal edit; the latest calculation and current goal remain distinct pieces of state.

Protein and fluid targets

Protein can use an explicit override or the app's automatic 0.7 g/lb relationship to weight. For an eligible 180 lb basis that is 126 g. The calculation source and missing-weight behavior need to remain visible rather than inventing a body weight when no reading exists. A target used for a past streak must be based on the applicable dated snapshot, not simply today's latest number.

`calculateFluidGoal` accepts custom or suggested mode. Custom mode validates the supplied mL and returns it directly. Suggested mode selects the source's 3,000 or 2,200 mL beverage reference, then adds the chosen activity allowance of 0, 250, 500 or 750 mL. For a 2,200 mL base and moderate allowance, the result is 2,700 mL. Total-water reference values are separate because food contributes water too; the beverage goal is not identical to total water intake.

Activity additions are product planning allowances, not measured sweat loss. A custom value is not forced through the suggested formula. Unit conversions preserve underlying canonical values so repeatedly switching UI units does not progressively round the stored target.

Saving goals without erasing unrelated edits

The goals repository retains the raw baseline it read. When saving, the mutation communicates changed fields and their expected prior values. If the phone changes fluids while the browser changes calories, unrelated fields need not conflict merely because both belong to `profiles`. However, the numeric calorie target and its associated calculation metadata can form a coupled change so the app does not save a number with the wrong explanatory snapshot.

The SQL goal path delegates to dated-goal logic and writes appropriate history. Suppose the calorie goal is 2,200 on Monday and changes to 2,400 on Thursday. Recomputing Monday's result against 2,400 would change the meaning of historical success. An effective-day snapshot lets the streak engine use Monday's target for Monday and the new target when effective. The update is more than assigning one current scalar.

The helper also retains calculation requests and results so an uncertain save can resume its original operation ID. Previewing a result, accepting it and subsequently editing a manual target are different actions. Tests cover conversion equivalence, saved comparisons, loss/gain arithmetic, coupled metadata, unrelated concurrent changes and helper draft preservation. They establish consistency with implementation assumptions rather than the medical adequacy of those assumptions.

---

17. Summary, charts, History and derived streaks

Summary is a composition of independent sources: goals, vitals, food, fluids, calendar, training, streaks and quick actions. Fetching them as one giant request would make the entire screen wait for the slowest source and fail if any source failed. Sustain instead uses keyed resources and independent loading/error states. [C18]

Caching without allowing old results to win

A **cache** is a retained copy used to avoid repeating work and to show something useful while fresher information loads. The resource store keeps entries by account and relevant parameters such as local day, timezone, month and configuration. A cache key is part of correctness: if two accounts share a key, a fast interface can become a privacy bug; if two months share a key, an older calendar response can replace the selected month.

`useResource` connects a component to an entry. Concurrent requests for the same entry reuse in-flight work. If an invalidation occurs while a read is running, the store suppresses stale publication and requests fresh

data as needed. **Invalidation** means marking the copy as needing a new read; it is not the database mutation itself.

Consider a user switching September to October while September's request is slow. The selected month/key changes immediately. September can complete for its own entry, but cannot be published as October. Similarly, an accepted nutrition save invalidates relevant food/calendar totals, and active subscribers refresh. Focus, foreground and day-boundary checks provide additional refresh opportunities. This is not a realtime subscription to every remote change.

Cached content remains visible during a failed background refresh rather than being replaced by an empty total. Initial empty vitals can wait for synchronization to settle so the screen does not falsely present an absence before cache/import work resolves. A skeleton means loading; an error means unavailable; a measured zero means known zero. Those states should remain distinguishable.

Query complete data before calculating

Summary repositories collect pages for records needed by the selected day/window. Training needs both sessions and sets, and food totals require all relevant entries rather than the first API page. Pagination tests use capped synthetic pages to reveal code that assumes one response contains everything.

Some reads use **keyset pagination**: order by a stable key, remember the last key and request keys after it. This avoids offset drift from rows inserted earlier in the order, although it still does not make multiple queries a frozen snapshot. Other queries use offsets and increment by actual returned length until an empty page. Export currently has a different short-page assumption, discussed later; pagination guarantees must be evaluated per path.

The selected architecture derives most visible aggregates on demand. It is straightforward to correct a historical entry and then recompute. It can become expensive as each user's history grows, particularly if full lists are collected repeatedly. A stored daily aggregate would reduce read work but would need transactional or asynchronous maintenance for creates, backdated edits and deletions. A fast but stale aggregate is a correctness tradeoff, not a free optimization.

From raw readings to a chart

The vitals domain sorts and filters live samples, deduplicates equivalent readings within its window and groups them into range buckets. Manual readings can take precedence in duplicate display; the underlying history still retains source records. Blood pressure uses a shared correlation ID where possible, with a limited legacy pairing fallback. Pulse association is bounded by time rather than assuming every nearby pulse belongs to every BP reading.

The D/W/M/6M/Y views use fixed spans of 1, 7, 30, 183 and 365 days. Buckets use local hours/days, Sunday-based weeks or months depending on range. A plotted average is a statistic derived from several real samples, not a new measurement. If two weights in one bucket are 180 and 182 lb, the bucket can display 181 even though no scale recorded exactly 181.

The curve uses monotone cubic interpolation with slope limiting to avoid artificial overshoot between values. A naïve smooth curve can dip below both neighboring readings or rise above them, visually suggesting a nonexistent excursion. Smoothness is presentation; neither the curve nor the stable full-data vertical scale makes sparse data more certain. The UI provides text readings and date controls so inspecting values does not depend entirely on gestures.

The browser regression for repeated diagonal gestures remains unresolved: date text did not change under the synthetic touch sequence. Another gesture test passing does not cancel that evidence. Native panning, screen scrolling, VoiceOver and larger text require their own tests.

Streaks are a dated query, not simply a counter

An incrementing streak counter is easy until someone backdates a workout, changes a goal or deletes a meal. Sustain derives active goal-habit results from records, effective rules and goal snapshots. States include met, open, not met, unknown and not scheduled. Today's result can remain open because the day has not ended. Missing coverage should not be represented as a confident failure or success.

Training can use weekly scheduling while calorie/protein/fluid habits apply their own dated conditions. A calorie day with no meaningful logged intake must not earn success merely because zero is below a target. A changed target should apply from its effective date, preserving past interpretation. Retired logging/reminder/PR habits are filtered from the active system even though historical schema remains.

Widget layout and enabled choices are account-scoped local preferences. Editing previews the real widget components, then Save commits the layout; Cancel discards that edit state. Reordering a widget is not changing its underlying data source. A remote goal can synchronize to another device while its widget arrangement stays local.

Tests cover late responses, midnight changes, independent failures, full pagination, layout persistence and dated streak rules. They do not establish low latency with a decade of history. Measure query duration, payload bytes, record counts and client calculation time before choosing stored aggregates, narrower windows or a different cache library.

18. Reminders: local scheduling and safe navigation

A reminder has three representations: the user's saved configuration, the app's desired schedule and the requests currently registered with iOS. They can disagree after a permission change, partial failure or process interruption. Sustain reconciles them rather than assuming one successful Save means every future notification will arrive. [C19, C20]

Routine preferences cover meals, fluids, BP, weight and workouts. Custom reminders include medication/supplements and configurable times/recurrence. Definitions, completion information and routine preferences are stored locally per account. They are not cloud cron jobs and are not automatically mirrored to another device.

From configuration to OS requests

`saveRoutinePreferences` or `saveCustomConfiguration` persists the relevant local choice and runs the serialized scheduling service. `notificationPermission` uses Expo Notifications to inspect permission and requests it only when appropriate. The native adapter is explicitly iOS-enabled; the browser provides guidance and synthetic behavior rather than delivering native reminders.

The schedule model turns settings into `ScheduleRequest` values. Each has a stable ID, owner, source/group, title/body, data payload, trigger and fingerprint. A **fingerprint** captures the request content so reconciliation can tell whether the same ID now means a changed time/message. Stable IDs make replacing/canceling a request deliberate rather than accumulating a new random notification on every settings save.

The actual adapter calls `Notifications.scheduleNotificationAsync` with an identifier, content and a calendar trigger. For a repeating daily/weekday request it supplies hour/minute and optional weekday. For dated occurrences it includes year/month/day and `repeats: false`. It lists current OS requests using `getAllScheduledNotificationsAsync` and removes selected IDs with `cancelScheduledNotificationAsync`.

These are on-device calls. They do not contact a Sustain reminder server or require an APNs device-token registration path in this implementation. iOS owns eventual presentation, user settings, focus modes and delivery behavior. The app can inspect registered requests but cannot force the operating system to show a banner at an exact visible moment under every device condition.

Reconciliation and finite horizons

The service compares desired requests with OS requests, schedules/verifies replacements and then removes obsolete entries. Its capacity plan uses a 60-request budget and a small temporary replacement allowance. The order protects existing schedules if a replacement fails; deleting all old requests first would leave no useful schedule after a partial error.

Explicit disable still cancels the applicable requests even if an unrelated capacity issue blocks other additions. A failed enable/save reports the problem instead of claiming every desired request exists. Generic previews are the default, reducing health detail visible on the lock screen unless the user opts into more detail.

Some configurations use repeating calendar triggers. Workouts use a rolling 28-day horizon, and custom once/interval/future-start/exception cases use a bounded list of upcoming occurrences, including the next 14 where configured. Those finite schedules must be replenished when reconciliation runs. A user who never opens the app again can outlive a finite schedule window; there is no hidden always-running JavaScript process to refill it.

The distinction between a local calendar time and a UTC instant matters during travel and daylight-saving changes. "Every day at 8 AM" normally expresses local intent, whereas a one-time occurrence represents a chosen dated event. The model and adapter convert local components accordingly, but actual OS behavior across DST, timezone changes, permissions and device capacity remains a required physical-device test.

A tap is data, not permission

The native observer captures notification responses, including launch-time responses. The payload carries version, account and allowlisted destination information. The navigation model validates it, deduplicates seen responses and stores a pending intent with a 30-minute lifetime. Arbitrary route text from a notification is not blindly executed.

ReminderNotificationObserver waits until startup capture, authentication, setup, privacy unlock and foreground requirements are satisfied. If the user signs into the wrong account, the intent must not open another user's logger. If setup is incomplete, onboarding takes precedence and the valid intent can resume afterward. Captures are serialized so a slower old validation result cannot replace a later tap.

For example, a breakfast tap arrives while signed out. The app saves an intent, opens login, then resolves entry directly to the Food logger after successful authentication/setup checks. If an unfinished dinner draft already exists, the meal-prefill rules preserve it rather than overwriting it with breakfast. A stale/canceled intent is discarded. Opening the screen never inserts a meal or awards completion.

`restToday` stores the local-day workout preference and affects today's schedule. It does not create a completed workout or a streak success. Explicit logout cancels the intended outgoing routine scope and pending navigation; account deletion performs broader owned-notification cleanup, including relevant delivered items. These cleanup scopes are intentionally different.

Local reminders versus server-driven notifications

Local scheduling works without a server sending each message and keeps reminder names/settings device-local. It fits personal routines and reduces operating cost. It also lacks centralized delivery observability and effortless cross-device editing. Server-driven notifications would require device tokens, account/token lifecycle, push credentials, scheduling workers, timezone rules, retries, provider feedback and preferences synchronized to the server.

Even that alternative would not guarantee visible delivery: APNs acceptance and iOS presentation are separate outcomes. A server scheduler becomes attractive for remotely triggered events, shared account schedules or operationally measured replenishment needs. It is not automatically more reliable simply because it runs in the cloud.

Pure schedule/reconciliation tests exercise recurrence, permission/capacity failures and replacement behavior. Browser tests exercise settings, pending navigation and draft protection with adapters. They do not establish notification delivery, cold launch, badges/sounds or OS-specific recurrence on an iPhone. Those limits belong in both a release checklist and an interview answer.

---

19. Progress photos and the boundary of a transaction

An uploaded photo combines a file and a structured relationship: this image belongs to this account and this weight reading. PostgreSQL can enforce the relationship, while object storage holds the bytes. The app must coordinate both, because no single transaction covers the file upload and the application metadata insert. [C21]

Selection and processing

`selectProgressPhoto` uses Expo ImagePicker. Camera selection requests camera permission; library selection uses the platform picker. Canceled selection returns no photo. The app requests an image, avoids requesting EXIF metadata, and passes the selected asset into processing rather than uploading the original immediately.

The image module uses `manipulateAsync` to resize/re-encode JPEG, limits the longest dimension to 1,440 pixels, targets a smaller compressed size and enforces a 2 MiB maximum. It then calls `stripJpegMetadata` and handles app-owned temporary files. This reduces upload size and removes metadata such as location from the prepared copy. It does not delete the user's original photo-library asset.

Encoding alone and `exif: false` are not treated as an absolute guarantee that every unwanted marker is gone; explicit JPEG metadata stripping provides another concrete step. Tests use synthetic JPEG structures and image-model cases. Actual picker, orientation, cancellation and native temporary-file behavior still need device verification.

Upload first, record second

`uploadProgressPhoto(userId, photo, weightSampleId, entryDay)` validates byte size and invokes `runMutation` under an entry-specific slot. The frozen request contains an upload payload and metadata row. It constructs an immutable object path using owner, local day and a new ID, such as `U1/2026-09-25/P1.jpg` in this abbreviated example.

`sendMutation` recognizes photo creation, checks the path owner prefix and calls `require_active_session`. It uploads JPEG bytes with `supabase.storage.from("progress-photos").upload(path, bytes, {upsert: false, ...})`. A 409 for that frozen path can be tolerated during retry because the same upload may already exist. The path should not be reused for unrelated content.

It removes the upload-only field from the SQL payload and calls `commit_health_mutation` for the metadata. The database verifies an owned live weight parent and serializes the per-entry quota. Counting photos only in the client would allow two devices to race past the three-photo limit; the database lock/quota check handles the authoritative decision.

After accepted metadata, the sender calls `createSignedUrl(object_path, 3600)`. The returned one-hour URL lets the image view fetch a private file without making the bucket public. It is a bearer capability for that object during its validity period; avoid putting it in logs or treating it as a permanent export identifier. Viewing later may require issuing another URL.

Failure windows are real

If upload fails, metadata has not yet been inserted by this path. If upload succeeds but metadata is definitively rejected, the current code has no compensating object removal in that branch. The file can be orphaned. If metadata succeeds but signed-URL creation fails, the app retains an uncertain/recoverable operation even though the record exists, and retry can obtain the accepted receipt and attempt the URL step again.

Photo deletion runs the reverse logical handoff: the versioned metadata deletion is accepted and receipted, then the client removes the returned object paths from Storage. If byte removal fails, retrying the same operation returns the recorded paths so cleanup can continue. The app does not claim that metadata deletion automatically removed the file.

Soft-deleting a weight reading also does not automatically delete all associated bytes. The metadata can remain while normal live-weight views no longer surface it. Full account deletion enumerates owned storage, including objects without a matching application photo row, but that is not an ongoing orphan-cleanup service.

Better alternatives and their costs

One proposed improvement is compensation: remove the newly uploaded path when metadata is definitively rejected, with its own retry record if deletion fails. Another is a staged-upload lifecycle: create an authorized pending metadata reservation, upload to a reserved path, then finalize; expire unfinished reservations with a reviewed cleanup task. The latter makes ownership/status explicit but adds states, expiry rules and background operations.

Neither alternative creates a distributed transaction. The design still has to tolerate the process dying after each step. Verification should deliberately fail upload, metadata insert, URL signing and object deletion independently, then check both tables and Storage using disposable data. A happy-path screenshot proves almost none of those recovery properties.

---

20. Export: collecting, packaging and sharing account data

Export is a read operation with a privacy-sensitive output. It collects cloud rows and selected device state, builds portable files and hands a copy to the user. It is not an account restore system, and the archive is not automatically synchronized back if edited. [C22]

The repository enumerates owned health tables, reusable foods/recipes/drinks, photo metadata, planner records, setup, streak state, processing choices and usage records. It also loads the account/profile and allowlisted device data such as drafts, local vitals/pending changes and preferences. The allowlist excludes session tokens, biometric secrets and deletion capabilities. Explicit categories make it easier to notice omissions when a new store is added.

`assertAccount` surrounds collection and asynchronous work. Without it, an export started by Alex might finish after another account signs in and be presented in the wrong context. The filter `eq("user_id", userId)` helps constrain requests, while RLS remains the server boundary. Local data loaders must independently scope their keys because PostgreSQL policies do not govern AsyncStorage.

Pagination and consistency limits

`loadRows` requests ranges of 1,000 and stops when the returned page is shorter than 1,000. This works only if a nonfinal page can actually return that many matching rows. If the server caps responses at 500, the first 500 looks final and the export can silently omit later rows. This is a conditional source finding; no truncation of a live export was performed or asserted in the review.

A proposed correction is to use validated keyset paging or advance by actual returned rows until an empty page, with tests that deliberately cap responses below the requested size. Counts/checksums can help verify coverage. Offset pagination still needs care under concurrent edits; changing the stopping condition alone does not establish snapshot consistency.

Tables are fetched at different times, sometimes in parallel. A workout might be edited between reading sessions and sets. The archive therefore represents a collected view, not one database-wide transaction snapshot. A future server-side export job could read under a suitable snapshot and produce a bounded archive, but would add job status, secure download expiry, storage cleanup and operating cost.

Files and CSV safety

`exportTextFiles` produces README text, a manifest of category row counts, full structured JSON and one CSV per dataset. CSV is convenient for spreadsheets but flattens object values as serialized text. JSON preserves arrays/objects more naturally. A UTF-8 marker and correct quoting help common spreadsheet tools read the CSV.

Text beginning with `=`, `+`, `-` or `@`, including after invisible controls, can be interpreted as a spreadsheet formula. `csvValue` prefixes dangerous string values with an apostrophe and quotes delimiters/newlines. This protects an exported note or food name from being treated as executable spreadsheet content. Numeric negative values remain numeric; the guard applies to strings.

Photos are optional because they can dominate size and time. On native, `shareHealthDataExport` uses fflate's streaming ZIP interface to write archive chunks to an Expo File handle in the app cache. It downloads photos one at a time, reports progress, closes the handle, asserts the account and calls `Sharing.shareAsync`. A finally block attempts to close/remove the temporary archive afterward.

Native streaming avoids collecting the final ZIP as one giant byte array, but the database datasets and generated text still materialize in memory. On web, the implementation accumulates file buffers, calls `zipSync`, creates a Blob/object URL and clicks a download link, then revokes the URL. Large histories/photos can therefore stress memory differently on the two platforms.

If a read or photo download fails, the workflow reports that it could not create a complete export instead of intentionally omitting that failed category. However, the pagination assumption can evade such an error because the request itself succeeded. This illustrates why transport success and data completeness need different tests.

The archive may contain sensitive health information and copies of local pending state. Once the user shares or saves it elsewhere, deleting the account cannot retract it. Tests cover local-key allowlists, CSV formatting/formula safety, account guards and browser export controls; they do not certify every external share destination's handling or very large native archives.

21. Account deletion as a resumable protocol

Deleting an account is harder than deleting a row because the process destroys the ordinary credentials that could authorize a retry. It also crosses object storage, PostgreSQL/Auth and device-local state. Sustain introduces a separate continuation capability so the user can check and finish deletion after normal sessions are revoked. [C23]

Prepare before destroying the route back

The deletion UI requires the current password and deliberate `DELETE` confirmation. Before asking the server to prepare, it generates and durably stores a random continuation token together with the account identity in protected local storage. It first probes status, so recovery of an existing request is distinguished from starting a fresh one. The raw token is a secret capability, not a display identifier.

`requestDeletion` intentionally uses direct `fetch` rather than only the normal signed-in function helper. It posts to `${supabaseConfig.url}/functions/v1/delete-account`, sets JSON content type and the public project key, includes a bearer token when available, and sends `action` plus the continuation token. `prepare` also carries password and confirmation. A 20-second AbortController timeout bounds waiting; it does not reverse server work.

A source-shaped teaching request is:

```
{
  "action": "prepare",
  "token": "<fresh secret continuation capability>",
  "password": "<current password supplied for this operation>",
  "confirmation": "DELETE"
}
```

The server requires independent account/password verification for initiation. It hashes the token and stores the hash in `account_deletion_jobs`. Normal authenticated clients cannot select those rows through the Data API. Completion receipts can outlive the Auth user because that table's lifecycle is deliberately separate.

Stop new writes, then remove data

Preparation and cleanup use account-specific database coordination. The prepared deletion state blocks application access/writes through restrictive policies and guards, including reviewed service-side paths. It revokes sessions and biometric credentials. This matters because removing existing data while allowing another device to keep inserting new records could make deletion never converge.

The cleanup handler enumerates actual owned Storage objects by the supported ownership/path relationships, in batches of 100 with bounded work per invocation. It removes objects before deleting Auth. Merely deleting `progress_photos` rows would miss orphan uploads. If a batch or limit prevents finishing, the job remains incomplete and can continue with the same token.

After Storage cleanup, the handler deletes the Auth account, letting declared database cascades remove owned rows. The finish routine checks the absence of Auth, relevant owned records and files before declaring completion. It then retains a minimal completed receipt without the original account identity. A lost final response can be checked using the same capability without pretending the user's expired/revoked access token still works.

State transitions and failure examples

Deletion is an explicit state machine

Saved locally Token exists before request	→	**Prepared / running** Writes blocked, files/Auth being removed

→

Verified complete
Receipt permits final confirmation

Cancellation can seal a request that has not started; it cannot restore records after deletion begins. Errors retain the continuation path.

If the network fails before prepare reaches the server, the local token still exists. A status probe can discover that nothing began. Cancellation can seal that token so a delayed prepare cannot later restart a canceled operation. If storage deletion fails after preparation, the user can resume without reauthenticating through the now-revoked normal session. If Auth deletion succeeds but the final response is lost, the completion/status route remains available.

The root AccountDeletionGate reads the pending marker and prevents ordinary use from quietly continuing through an unfinished cleanup state. It offers recovery and permitted cancellation behavior. Signing out from that screen does not casually erase the only continuation capability. Once server completion or confirmed cancellation is known, the app can clear the marker appropriately.

Local cleanup drains health-sync/vitals work, removes account-owned cache/outbox/drafts/preferences and credentials, cancels relevant notifications and signs out. Draining matters: a previously started task must not rewrite a just-cleared key after deletion. Clearing mounted draft state also prevents an old component from repersisting stale input.

What the protocol does not promise

There is no rollback that recreates a deleted Auth account, photos and all cascaded rows after cleanup has begun. This is a converging workflow, not one giant distributed transaction. Its continuation token must be stored safely and never logged. Losing that token mid-cleanup is a recovery/operations issue that needs an explicit support policy for public release.

Another offline device may retain local data until it runs and observes revoked access/cleanup behavior. Exports already shared elsewhere are separate copies. Provider retention and backups have separate lifecycles. The application's successful completion means its defined active storage checks passed, not that all copies anywhere in the world disappeared instantly.

Mock handler tests cover Storage failure, Auth failure and premature completion prevention. Isolated SQL tests exercise deletion write guards, ownership cleanup and completion checks. This review did not delete a real account or exercise deployed Storage/Auth with a disposable live account. That integration check remains necessary before a public release claim.

An alternative would enqueue deletion entirely server-side and let a worker retry until finished, with a status endpoint and operational alerts. That can reduce dependence on the app remaining available, but still needs secure job authorization, bounded retries, cancellation semantics and accurate completion criteria. The present resumable handler is smaller; its operational limits should motivate improvements if real failures show the need.

---

22. Settings, privacy boundaries and operational ownership

Profile organizes settings into focused destinations rather than one giant form. Cloud-owned choices such as profile names, setup units and goals differ from local choices such as appearance, widget layout and reminder definitions. Reading a value on one device does not establish that another device will see the same preference. [C02, C08, C24]

Appearance supports device/light/dark using a local setting. The provider resolves the effective scheme, shared theme values determine component colors and a rerender changes presentation. This setting does not require a health-record mutation. Motion preferences similarly change animation without changing whether a save or authentication step is authorized.

Privacy controls use explicitly named scopes. Clearing unfinished drafts removes/reset those forms while retaining unresolved saves that may already exist on the server. Clearing saved AI data removes the targeted planner records/profile through protected server logic; it does not delete

completed meals/workouts or imply every processing-choice/usage receipt and provider copy has been erased. Export and full deletion are separate operations.

The current legal/privacy pages are shared-source drafts accessible both signed out and through Profile. Public publication and ordinary production builds are gated on unresolved release fields rather than presenting unfinished operator/contact/territory decisions as complete. This is a deployment safeguard in repository code, not a legal opinion or certification.

Native preparation includes a plugin excluding private app directories from backup before React starts, accurate permission descriptions, no unused microphone permission and development-only broad transport exceptions. Failure to establish the configured backup protection is intended to fail closed. The compiled result and restoration behavior must still be tested on an actual device/archive. Configuration text alone cannot establish that an installed older binary has the new behavior.

Data minimization means sending/retaining only information needed for the operation. The workout planner's training-only optional history, generic reminder previews, processed metadata-stripped photos and bounded diagnostics are concrete examples. It does not mean no health data leaves the phone: accepted logs are cloud-backed, selected AI inputs go to a provider, and export explicitly creates a portable copy.

For a one-person project, the owner also acts as operator: responding to failed deletions, revoking compromised credentials, rotating provider keys, verifying restores, reviewing logs and supporting users. Supabase reduces hosting work but does not supply the product's support policy. Before inviting public users, decide who receives support/privacy requests, what logs are retained and how an incident is investigated without copying private records into uncontrolled tools.

Tests cover parts of these controls and synthetic accessibility paths. There is no claim here of legal compliance, native accessibility certification or verified provider contractual suitability. Those questions require their own evidence. The engineering lesson is to make boundaries observable and scoped, so an operator can explain what each control actually does.

23. Design choices, credible alternatives and when to change

A strong design explanation begins with a requirement and a failure mode, then compares alternatives against them. "It is scalable" is not an explanation. "It keeps parent/child writes in one database transaction while avoiding a separately hosted API process" identifies mechanisms and a tradeoff. The following comparisons are retrospective engineering analysis unless an ADR is explicitly cited. The owner's stated personal-use, one-person maintenance and React/TypeScript learning goals are facts; undocumented historical experiments are not.

React Native and Expo versus SwiftUI, Flutter or web

The simplest iPhone-only native choice could be SwiftUI with Swift models and direct Apple-framework calls. SwiftUI offers Apple's declarative UI framework; a native implementation would avoid needing a JavaScript bridge for HealthKit and could align closely with Apple platform conventions. It would require learning/maintaining Swift and native tooling, and a browser version would be a separate UI effort. For an app dominated by sophisticated Apple-only interactions with no React learning goal, that could be preferable. [R11]

React Native preserves the React component/state/hook model while rendering platform UI. Expo supplies project/build integration and common modules. Sustain can share TypeScript models, domain tests, validation and much UI across iPhone and browser preview, which is useful for rapid iteration and the owner's learning goals. React state, component composition and asynchronous data-flow skills transfer to React web. Native permission prompts, app lifecycle, Keychain, signing, gestures and mobile keyboard behavior do not become ordinary web problems simply because the language is TypeScript. React's state documentation explains the shared component-memory model. [R09]

The cost is dependency compatibility and bridge behavior: a JavaScript update cannot add a missing native HealthKit module, and browser tests cannot reproduce the installed binary. Expo Go is insufficient for this native dependency set. Native build/debug expertise remains necessary. The right defense is not "one codebase means no native work" but "shared domain/UI work reduces duplication while native adapters isolate unavoidable differences."

Flutter is another credible cross-platform alternative. Its Dart framework, rendering approach and platform channels/plugins provide a different way to build shared UI and call native services. A Sustain implementation would still need an Apple Health integration, durable local storage and the same backend correctness rules. It might suit an owner who preferred Dart/Flutter tooling and its widget/rendering model, but would not directly serve the stated React/TypeScript learning objective. No performance comparison between these implementations was measured here. [R10]

A web application could deploy quickly and avoid app-store binary distribution for UI changes. It could implement forms, cloud history and many calculations. It could not use Sustain's HealthKit/Keychain/iOS notification adapters unchanged. A native wrapper could add bridges, reintroducing native code and release work. If the product became primarily a desktop analytics dashboard, web-first would become more attractive; for current iPhone health integration, native access is a real requirement.

Supabase versus a custom backend

For personal use, Supabase supplies a managed identity/database/storage/function boundary without requiring a separate always-running application server. The app can read permitted tables with a user token, call SQL functions for transactional writes and use Edge Functions only where needed. This reduces the number of deployed components the owner operates. ADR-001 documents hosted Supabase without requiring Docker. [C25]

A custom backend could expose routes such as creating a meal, updating a workout and reading a summary. Each route would verify tokens, derive account identity, parse inputs, enforce permissions, open database transactions, map errors and return a versioned response. It would also need hosting, TLS/routing, connection management, deployment/rollback, logs/metrics, secret rotation, provider timeouts and monitoring. If replacing Supabase Auth/Storage too, password flows, email delivery and file lifecycle would add more responsibility; alternatively the custom API could retain those managed services.

The benefit would be a centralized application API with explicit contracts, reusable middleware and greater freedom to move business logic between databases. That can become preferable when several client types need an API independent of tables, workflows span many non-database systems, or a team already operates an established backend platform. For Sustain's current owner and workload, duplicating that service layer everywhere would add maintenance before proving a need.

Self-hosting Supabase would preserve some APIs but move upgrades, backups, secrets, service availability and capacity onto the operator. It is not equivalent to eliminating vendor dependence for free. PostgreSQL data and SQL are portable in principle, but Auth, Storage, Edge deployment conventions and client integration still require migration work. A credible portability plan exports data, documents contracts and rehearses restore/migration rather than claiming there is no coupling.

PostgreSQL and SQL functions versus documents and API logic

Workout sets, owned parent relationships and dated history naturally benefit from relational constraints and queries. PostgreSQL can enforce same-owner foreign keys and commit multiple rows atomically. A SQL function keeps validation and changes close to the rows, avoiding multiple network round trips while a transaction is open. The transaction's all-or-nothing behavior comes from PostgreSQL; defining the right transaction boundary is Sustain's responsibility. [R12]

A document-oriented design could store each workout and its sets together and make whole-workout retrieval simple. Queries across sets, ownership constraints across documents and partial updates would need the chosen store's supported indexing/transaction features or application logic. Such databases can

be entirely appropriate; the point is to examine Sustain's queries rather than declare one category universally superior. Mixed relational columns and recipe JSON already capture some document-like convenience without abandoning SQL.

Putting all business logic in an HTTP application server would make it easier to use a single language/testing style for many rules. It would still need database constraints and transactions: an API-only owner check can be bypassed by another privileged writer or a coding error. Putting everything in SQL can make network/provider workflows awkward and turn one function into a large dispatcher. Sustain divides those responsibilities, accepting the need to understand both TypeScript and PL/pgSQL.

The large mutation dispatcher is an intentional central boundary with a maintenance cost. Adding a supported table requires reviewing allowlisted fields, ownership, create/update/delete behavior, receipt response and tests. A future decomposition into smaller internal SQL functions could improve readability while retaining one externally coherent transaction. Replacing it with many independent client calls would not preserve the same atomicity.

Local-only, cloud-backed and broader offline-first designs

A local-only app could store everything in SQLite. It would be simpler to reason about a single device and could work without internet, but recovery after device loss, multiple devices, server-held AI credentials and account export/deletion expectations would need different answers. Device backups are not the same as an intentional cloud account model. For strictly private single-device tracking without remote AI, local-only might be enough.

Sustain uses a cloud-backed model with targeted offline durability. Vitals have a transactional outbox; other features use drafts and pending operation records. That concentrates synchronization complexity where it has clear value. The drawback is inconsistent offline capabilities between features, which the UI and documentation must communicate.

A broader offline-first design would need outboxes for meals, labels, recipes, photos, goals and workouts, plus dependency ordering, conflicts and merge policies. For example, an offline meal could reference a newly created private label and a photo upload that cannot finish yet. It would need durable dependency identities and careful cleanup if the user deletes the draft before synchronization. A generic queue library cannot decide those domain semantics.

Broader offline support becomes compelling if real users need reliable logging during long disconnections across those features. Otherwise it can multiply states and migrations for little daily benefit. The decision should depend on recorded failed/offline workflows, not the appeal of an architectural label.

Versions, locks, merging and last-write-wins

Last-write-wins accepts whichever update reaches the server last. It is simple for harmless preferences, but it can erase an earlier intentional health-record edit without explanation. Version checks identify stale data and allow the user to choose. Locks serialize active changes but cannot detect a stale form that has been open for hours; using both solves different problems.

Automatic field-level merging works well when changes are independent, such as a display name and a separate fluid target. It becomes risky when fields are coupled, such as quantity and nutrient totals or goal number and calculation basis. Sustain uses goal baselines for selected independent fields and versioned replacement for records/workouts, accepting more conflicts in exchange for clearer consistency.

A **CRDT**, conflict-free replicated data type, defines merge operations that converge without central coordination for suitable data structures. It is not a universal answer to "two health edits conflict." Concurrently setting one BP reading to 120 and 135 still needs product meaning. Collaborative text or additive counters can fit CRDTs better than replacing a measured value. Adopting them would be preferable only for a clearly specified collaborative/offline merge requirement.

Derived values, snapshots and stored aggregates

Sustain copies consumed nutrition values so historical meals remain stable; it derives charts and streak results from accepted records and dated goals so edits recalculate. These are compatible choices: preserve the facts/assumptions that define an event, then compute views from them.

Stored aggregates could make Summary fast for very large histories. They require invalidation or transactional updates whenever a source row changes, including moving a record between dates or deleting it. A repair/recompute process is needed when code changes or an update was missed. On-demand derivation is slower but easier to inspect. A measured rise in read volume/latency would justify incremental daily aggregates with comparison tests against the raw calculation.

A fully event-sourced system would keep immutable domain events and rebuild views. It provides detailed replay/audit potential, but complicates event versioning, correction, erasure, compaction and read-model maintenance. Sustain's mutation receipts and vitals cursor are not a complete event-sourcing system; they solve narrower retry/sync needs. Claiming event sourcing because a receipt exists would misdescribe the architecture.

Existing state management versus additional libraries

The custom resource store gives explicit keyed reads, in-flight deduplication, retained data and invalidation. React context handles shared authorities, while forms own editable state and draft persistence. This is small enough to follow without an additional global application-state framework, but the owner must maintain subtle stale-result and invalidation behavior; the workout event mismatch shows that cost.

A query-cache library could standardize caching, retry and request invalidation conventions and reduce bespoke resource code. It would not provide durable mutation receipts, offline conflict semantics or account-safe persistence automatically. A centralized state library could make broad cross-screen transitions more inspectable, but can add indirection for simple local inputs. Choose one when measured maintenance pain or feature complexity exceeds migration/testing cost, not because a framework list looks incomplete.

Decision triggers, after the reasoning

Present choice	Reason to revisit
Expo/React Native + native adapters	Native integration complexity dominates shared-code/learning benefits.
Managed Supabase services	Established team platform, contractual requirements or workflow needs justify operating more infrastructure.
Targeted offline vitals	Users demonstrably need equally durable offline meals/workouts/photos.
Per-account write serialization	Lock-wait measurements show same-account contention limits useful throughput.
On-demand summaries	Representative histories exceed latency/memory budgets despite narrower reads/index tuning.
Local reminders/preferences	Cross-device control or remote-triggered events become required.
Custom biometric credential exchange	Maintenance/risk outweighs the intended process-session policy; a simpler supported session model meets the requirement.

Each row is a conditional decision, not a promise to migrate. A good interview answer can defend today's balance and state what evidence would overturn it.

24. From source code to a released iPhone app

Deployment means placing each program and configuration where its users can run it. Sustain has at least three separately released parts: the installed app, the database schema/functions and the Edge Functions. Public policy pages and provider configuration are additional operational artifacts. A Git push changes source availability; it does not deploy all of these parts automatically. [C26]

What a native build contains

The app's TypeScript/JavaScript is bundled with assets and executed by the React Native runtime. Native modules are compiled into an iOS application, along with bundle identity, entitlements, permission descriptions, icons and launch configuration. Signing associates that binary with an authorized developer identity/distribution method. A device must install the binary containing HealthKit, notifications and Keychain integration before those native calls can work.

Expo Application Services, EAS, can build a configured project remotely, which is useful from Windows where native iOS compilation is not performed locally in this workflow. A development build includes the app's actual native modules and development support; a production build packages a distributable application. Expo's official guide distinguishes these from Expo Go. [R03]

`apps/mobile/eas.json` defines development, preview and production profiles. Development uses an internal development client and `APP_VARIANT=development`; the dynamic config sets `Sustain Dev`, a development scheme and a separate bundle ID. Preview/production use the nondevelopment identity. Installing variants with the same identity can replace one another; profile names alone do not guarantee separate installed apps.

The effective config comes from workspace `app.json` plus `app.config.ts`, not an unrelated root file. Native changes such as a new library, entitlement, purpose string, icon/splash or backup plugin need a rebuilt binary. The current source sets `updates.enabled: false`, so it does not claim an over-the-air EAS Update rollout path is active. A development reload and a production update are different delivery mechanisms.

Configuration and secrets

The client requires public Supabase URL/key configuration and the appropriate callback scheme. Values exposed through `EXPO_PUBLIC_` are embedded in client code; they should be treated as readable by users of the binary. Provider keys and service-role credentials stay in server secrets, where the Edge handler reads them. Merely putting a key in an environment file does not make it secret if the build injects it into the app. [R13]

Separate environments should use deliberate app/backend/provider configuration so testing cannot accidentally mutate production. The browser review fixture uses a synthetic project URL and intercepts requests; its success does not prove the real environment variables, auth redirects or hosted function versions are correct.

Database and function rollout order

The current privacy contract requires new schema/functions and an updated client to agree. A practical rollout plan, not performed in this review, is to rehearse migrations against disposable data, deploy compatible database changes, deploy Edge handlers with configured secrets, then test the combined service with disposable accounts before distributing the matching client. Supabase CLI/database and function deployments are distinct operations. [R01, R08]

A migration can revoke direct-write permissions to make older unsafe clients fail closed. That improves enforcement but can break older installed clients. The product needs an understandable update-required path and a compatibility window where possible. **Expand-and-contract** is a common migration strategy: add a compatible new field/function first, migrate clients/data, then remove the old path after adoption. It is not always possible when an immediate security boundary must change.

Rolling back a mobile binary does not roll back database rows already written. Dropping a new column to mimic an older schema can destroy data. Prefer forward fixes and explicit compatibility analysis; rehearse restore before assuming backups solve a bad migration. This report has no evidence of a newly executed hosted restore or compiled-archive test.

Personal use, invited beta and public release

For personal development, use the isolated development identity and native client to verify the app on the owner's phone. Confirm permission denial, offline writes, credential recovery, notification taps and export. Preserve a reproducible source/configuration snapshot and understand which backend it uses.

For invited beta, TestFlight distributes uploaded builds to testers and collects feedback/crash context through Apple's process. It is a distribution/testing channel, not the app's database host and not equivalent to public App Store approval. Apple's current overview describes internal/external testing and build review requirements; use the current App Store Connect workflow rather than hard-coding future platform limits into a plan. [R07]

Before public release, resolve the operator/contact/countries/age/monetization and provider/legal review fields, publish reviewed policy pages, complete device/accessibility checks, establish support and incident procedures, and verify data export/deletion on disposable hosted accounts. The repository intentionally blocks ordinary production builds while its release fields/policies remain unfinished. Bypassing that gate would not complete the missing decisions.

App Store submission additionally requires the current metadata, privacy declarations and review process applicable to the release. This guide does not invent approval, eligibility or compliance.

No exact subscription/build/provider prices are quoted; service prices and requirements should be rechecked when choosing a paid plan or submitting.

Deployment verification should exercise the contract

A server returning HTTP 200 to a health check is insufficient. Verify that a fresh account gets profile/setup rows, a mismatched owner is rejected, an uncertain retry does not duplicate a meal, an old session is rejected, a photo follows its ownership policy, consent is enforced before AI use, and a disposable deletion removes files/Auth/rows and can confirm a lost final reply.

Then verify the installed binary separately: HealthKit authorization and imports, Face ID cancellation/rotation, foreground entry, actual notifications, keyboard insets, VoiceOver/Dynamic Type and backup behavior. Local iOS bundle export checks JavaScript/native configuration compatibility at a limited level; it does not replace compiling, installing and testing that binary.

---

25. Scaling and cost without speculative infrastructure

Three growth dimensions behave differently: more accounts, more simultaneous actions and more history per account. Adding accounts increases total records and provider usage. Simultaneous writes can create connection/lock pressure. A single long-lived account can make full-history reads, JSON cache writes and exports expensive even if no one else uses the app. Plan around these workloads rather than one undifferentiated user-count target.

A numerical example with explicit assumptions

Assume, solely for planning, 1,000 active accounts each create 12 health rows/day. That is 12,000 rows/day or roughly 4.38 million/year before receipts, versions, indexes and photos. This is not a measured Sustain load. If all actions concentrate in one hour, the average is about 3.3 new rows/second, but requests may batch rows or create additional reads/writes; peak bursts matter more than this average.

Assume 100 users each request two AI estimates/day. That is 200 provider requests/day. If one request averages I input tokens and O output tokens, daily inference cost is approximately `200 × (I × inputPrice + O × outputPrice)` with prices expressed per token. Image/tool costs and failed/repeated requests need separate terms. App quota limits bound one axis of demand but are not a verified account-wide currency cap.

For photos, assume 1,000 processed images averaging 500 KiB: raw file storage is about 488 MiB before backups, downloads or repeated exports. Repeated viewing creates bandwidth even if no new files are stored. These

estimates show which quantities to measure; they do not predict a bill or capacity without current plan/pricing and actual telemetry.

Measure the slow path, not only total screen time

Break a save into validation/local-persistence time, network latency, server authorization/lock wait, SQL execution and response/application time. Break Summary into query count, rows/bytes, client mapping/aggregation and render work. Record percentiles such as p50 and p95, where p95 is a duration at or below which 95% of measured requests fell. An average can hide a frustrating tail.

Useful operational fields include operation category, bounded error code, duration, returned row count and request correlation ID. Avoid recording health values, passwords, tokens or raw AI prompts just to get useful timing. Observability should explain failures without becoming an uncontrolled second health database.

If SQL is slow, inspect query plans and rows scanned with realistic disposable histories. If requests are numerous but each fast, batch related reads or reduce redundant refreshes. If client aggregation is slow, narrow windows or move a well-defined aggregate server-side. If large images dominate, optimize processing/download behavior before changing the database engine.

Likely pressure points in the present design

Per-account locking serializes writes for that account, which is usually acceptable for personal logging. Independent users need not share that same row lock. A heavy import or multiple devices on one account can wait behind each other. Measure lock wait before splitting locks by table/entity; narrower locks increase concurrency but require consistent ordering to avoid deadlocks and preserve cross-record rules.

Whole-account JSON cache/outbox state increases local parse/write cost with history. Normalize local rows/operations only after measuring device performance and designing a migration that preserves pending operations. Full training history and export can accumulate large arrays. Pagination bounds each response but does not bound the total memory used when the code collects all pages.

Summary's derived aggregates favor correctness and simplicity but may repeat large reads. Start with narrower selection/windowing and verified indexes. A daily aggregate table could then be introduced for the hot path, with tests comparing it against raw calculation after inserts, backdated moves and deletions. Keep a repair path rather than assuming aggregate maintenance cannot fail.

Receipts and tombstones grow over time. Their storage buys replay/deletion correctness. A compaction plan needs a defined retry/offline horizon, migration behavior for old clients and evidence that forgotten IDs cannot be replayed as new. Deleting them only because they look old can convert a space optimization into duplicate or resurrected records.

When a worker, cache or service split would help

A background worker could finish long exports/deletion cleanup or let AI results be retrieved after the client disconnects. It needs durable job IDs, authorized status/result reads, bounded retries, idempotent side effects and operational alerts. A queue helps schedule work; it does not make provider calls or files transactional.

A shared cache such as Redis might help repeatedly requested public/catalog data or a measured expensive aggregate, but introduces invalidation, access scope, hosting and failure behavior. The existing shared barcode cache already avoids some repeated provider calls using PostgreSQL. Adding Redis without measured database/provider pressure would create another state system to operate.

Microservices could isolate independently scaling/owned domains for a larger team. They would also replace local/database calls with more network failure boundaries, deployment contracts and tracing requirements. Nothing in this review establishes that a one-person tracker needs that split. The incremental path is measure, narrow reads, fix correctness gaps, improve tests/observability and only then introduce infrastructure for a demonstrated bottleneck.

---

26. Verification, known gaps and an improvement plan

Testing establishes behavior under specified conditions. A test that stubs Supabase can prove a screen preserves a draft after an injected error. It cannot prove the actual hosted RLS policy rejects the same request. Conversely, an SQL test can prove a function rejects a stale version without proving the native UI explains that conflict accessibly. Sustain needs both layers and integration evidence between them. [C30]

What ran for this review

`npm run check` passed lint, TypeScript, 256 app tests and 70 Edge tests. Lint catches selected static patterns; TypeScript checks declared type compatibility; runtime tests exercise code with concrete values and controlled failures. TypeScript types disappear at runtime, so Zod checks remain necessary for storage/network/provider input.

The maintained `check-onboarding-sql.mjs` harness applied all 37 migrations to isolated PGlite with platform stubs, then passed hardening, setup, streak/Summary, privacy and deletion regressions. PGlite exercises PostgreSQL behavior in an isolated environment but this single-connection harness does not establish actual overlapping-connection lock behavior. The older `check-hardening-sql.mjs` failed because its synthetic Storage table lacks `owner_id` required by a newer migration; do not treat that old fixture failure as proof of a hosted migration defect.

The full Playwright browser suite used synthetic account/API fixtures: 110 passed and two failed. Both failures reproduced unchanged in a focused rerun. One fills the signup email but finds it inactive instead of focused. The other dispatches repeated diagonal chart touches but observes unchanged empty-chart/date text. The root causes remain unresolved. Hiding a failing assertion or calling it flaky without evidence would weaken the review.

No app behavior was changed to obtain these results. Native device checks, real provider quality/latency, deployed schema/function compatibility, live disposable deletion and multi-connection/load behavior were not executed. Prior STATUS records for iOS export, native config and hosted inventory remain historical and explicitly separate.

Proposed improvements with acceptance criteria

Export completeness. Current short-page termination assumes a 1,000-row server response capability. Replace that assumption with robust paging and test against a fixture cap below the request size, including empty datasets and multiple pages. Verify exact IDs/counts and behavior under concurrent edits. Snapshot exports would require additional design; do not claim that a paging fix alone supplies them.

Photo orphan recovery. Add explicit compensation or staged uploads for metadata rejection. Test the process stopping after every handoff, a rejected fourth photo, revoked session, missing parent and cleanup retry failure. Success means no unmanaged object is left after the defined recovery window, while a legitimate previously accepted photo is never deleted by an uncertain retry.

Workout resource refresh. Map create/replace workout actions to `workout_sessions` or a typed event that affected resources recognize. Verify the already-mounted Summary/History updates after accepted saves without relying on route focus. Do not broaden the change into a new state-management framework before a small typed fix is evaluated.

Browser regressions. Reproduce focus with navigation/lifecycle timing recorded, and chart gestures with event coordinates/state transitions recorded. Determine whether app behavior, browser support or fixture timing causes each failure. Preserve the user's intended interaction; update expectations only when the requirement is demonstrably wrong.

Hosted privacy rollout. Rehearse and deploy the prepared migration/functions with a compatible client under separate authorization. Use disposable accounts to check denied/missing consent, minimal receipts, export categories and deletion cascades. Verify version/state from the hosted environment instead of assuming source deployment.

Device release evidence. Compile/install the current binary and exercise HealthKit, Face ID, notification cold launch, permission denial, accessibility, keyboard and backup protections. Record build identity and iOS/device versions. A native prompt screenshot alone does not prove cancellation, recovery and app departure behavior.

Make tests prove a claim

For retries, assert the same operation ID/request is sent after a lost response and that only one set of rows exists. For conflicts, use two distinct operation IDs with the same expected version and assert one cannot silently overwrite the other. For account isolation, test both reads and nested ownership references. For deletion, inject each failure and assert completion remains false until all defined checks pass.

For calculations, use independent expected arithmetic, unit-conversion boundaries, unknown values and dates near midnight/DST. Tests that merely repeat the implementation formula with the same mistake are weak. For UI, check reachable controls, retained input, visible error recovery and keyboard/focus state rather than only screenshots. For scaling, generate representative data and report workload, hardware/service configuration and percentile results; never infer capacity from unit-test speed.

CI currently runs `npm ci` and `npm run check` on Node 24. SQL and browser suites were additional local evidence in this review, not all automatic CI gates. Adding maintained deterministic SQL/browser checks to CI would reduce regressions, but requires controlling fixtures, execution time and native-versus-browser expectations. The quality goal is relevant repeatable evidence, not the largest possible test count.

27. Security and distributed-systems questions behind the features

An interview can move from a specific bug to a broader systems concept. The safest way to answer is to connect the concept to the actual boundary in Sustain, explain the assumption and give a counterexample. This chapter teaches the less-visible concepts that support those answers; it does not claim that every named technique is implemented.

Threat model: who or what can cause harm?

A **threat model** identifies assets, actors, boundaries and likely abuse. Assets include private health records, account access, uploaded images, provider credits and deletion capabilities. Relevant actors include another authenticated user, a malicious client bypassing the UI, someone with access to the phone, an attacker with a copied token and an operator accidentally exposing logs. Ordinary bugs and network failures can also violate privacy or correctness without an attacker.

The app cannot treat its own screens as trusted enforcement. An attacker can send the RPC directly with another user's ID or change a photo path. Server session/owner checks, RLS and same-owner foreign keys are therefore necessary. Client `assertAccount` guards a different risk: correctly authorized but delayed work being displayed or applied after the local user changes.

The service-role key can bypass ordinary access policies. Keeping it in an Edge Function avoids distributing it to clients, but makes that handler a privileged boundary. It must authenticate before acting, derive identity instead of trusting a supplied user ID, constrain fields and avoid sending raw provider error bodies to clients. The source review is not a penetration test or complete proof against every attack.

Authentication, authorization, encryption and hashing

Authentication establishes identity; authorization limits actions. Transport encryption protects bytes in transit between endpoints; it does not decide which authenticated account may read a row. Device Keychain protection governs a credential item; it does not encrypt every AsyncStorage draft. A signed URL authorizes temporary object access to its holder, which is why sharing it leaks a capability even if the bucket itself is private.

A **hash** maps input to a fixed representation that is used here for comparing secret capabilities without storing their raw value. Hashing is not reversible encryption. Random high-entropy device/deletion secrets differ from human-chosen passwords: password verification belongs to the Auth service and should not be replaced with a simple client hash. Do not claim the whole account is end-to-end encrypted; server-side queries and AI context access demonstrate that the backend can process the permitted data.

SQL injection and untrusted structured input

SQL injection occurs when untrusted text becomes executable query structure instead of a bound value. The SDK encodes ordinary filters/arguments, and server SQL must still constrain any dynamic table/field construction. Sustain's mutation function allowlists supported tables/fields; arbitrary caller-supplied table names must never become unrestricted dynamic SQL. A Zod parse helps establish shape/ranges but is not an authorization check.

AI prompt injection is a different boundary: untrusted notes/provider text can attempt to influence model behavior. Server-supplied identity, read-only bounded tools, narrow context and post-validation reduce what a manipulated model can do. The model does not receive a database mutation tool or authority to choose another account. These controls bound capabilities; they do not prove a model will always produce appropriate prose.

CORS, Cross-Origin Resource Sharing, controls which browser origins can read certain responses. It is not authentication and does not stop a nonbrowser client from sending a request. An `apikey` header identifying the public project is likewise not proof of a user's ownership. The bearer token and server rules do that work.

Consistency is scoped, not one global property

Within the accepted mutation transaction, related rows and receipt commit together. The phone's cache can lag behind those rows. A photo object can exist before its metadata. A second device may not refresh until focus or synchronization. Describing the whole system as simply "strongly consistent" or "eventually consistent" would erase important differences.

Sustain's local vitals can temporarily show unconfirmed user changes while remote state remains older. That is intentional availability for offline logging, with explicit pending/conflict states. Other features may require a successful network save before appearing in completed history. During a network partition, the app cannot both immediately confirm a remote commit and operate without contacting that remote system.

The **CAP** result concerns distributed systems under network partitions; it is not a rule that an application must permanently choose two product adjectives. In a Sustain interview, explain concrete behavior instead: a manual vital can be durably queued locally, while a server-owned goal write remains unconfirmed until contact returns. The conflict protocol defines what happens when disconnected edits meet later.

Timeouts, retries and at-least-once execution

A timeout means the caller stopped waiting, not that the callee rolled back. Retrying the same receipted mutation makes repeated delivery safe for that operation. Retrying a paid AI call can perform inference twice because it lacks the same result-retrieval protocol. Canceling a notification-edit screen does not automatically cancel already-registered OS requests unless the app's state machine performs that cleanup.

At-least-once delivery permits repeated attempts; the recipient must tolerate duplicates where necessary. **At-most-once attempts** avoid automatic retry but can lose work when an outcome is unknown. "Exactly once" needs a precise scope. Sustain can ensure one accepted database effect for a retained operation identity/request using receipts and transactions; it cannot promise one physical network transmission or one provider computation across every feature.

Backoff spreads retries over time to avoid overwhelming an unavailable service. Jitter adds randomness so many clients do not retry in synchrony. Those are useful potential network strategies, but this guide does not claim a universal exponential-backoff engine exists in Sustain. Before adding automatic retries, classify the operation as safe to repeat and decide which identity must remain fixed.

Locks, deadlocks and isolation

Transactions provide all-or-nothing changes; isolation governs what overlapping transactions can observe. A row/advisory lock creates ordering for participating operations. If two code paths acquire locks in opposite orders, they can deadlock: each holds what the other needs. PostgreSQL can abort a participant, but the application must handle the failure and retain correct retry identity.

The current account serialization simplifies reasoning by funneling related writes through a shared account boundary. It can lower concurrency for a heavy single account. Splitting it requires documenting a consistent lock order across mutations, deletion and quota/ownership checks. The review's sequential SQL harness verifies many invariants but is not evidence that every concurrent interleaving has been executed.

An optimistic version check avoids holding a lock while the person edits a form. The transaction lock is held only while validating/committing. Holding a database transaction open while waiting for a user or remote AI request would tie up resources and increase failure exposure. Keep external work outside the critical section unless a carefully designed protocol requires otherwise.

Durability, backups and recoverability

Durability means an acknowledged write persists under the storage system's contract. It does not mean the data cannot later be intentionally deleted, corrupted by a migration or lost beyond the provider's recovery guarantees. Backup availability and demonstrated restoration are different evidence. Define **RPO**, recovery point objective, as acceptable data loss in time, and **RTO**, recovery time objective, as acceptable restoration delay, before selecting a recovery procedure.

This review did not establish numeric RPO/RTO or execute a live restore. A useful proposed test would restore a disposable database/storage snapshot, verify record/file relationships and check that Auth/session/deletion state is handled deliberately. Restoring only SQL while losing private photos may not meet the product's recovery objective.

For unfamiliar questions, start with the invariant: what must remain true? Examples are "a set must belong to the same owner as its session," "one retry must not duplicate a meal," or "deletion must not be declared complete while owned files remain." Then identify which code/database/OS mechanism enforces it and the evidence that supports the claim. This method produces a defensible answer even when the interviewer changes the scenario.

28. Interview answers with deeper follow-up chains

The first-person answers below are practice language grounded in the reviewed project. Use them only to describe work and reasoning you can explain yourself. They are not claims of unrecorded production traffic, measured benchmarks or historical decisions you did not make. Start with a short answer, then follow the interviewer into the mechanism rather than reciting an entire chapter.

Tell me about your project

A natural opening: "I built Sustain for my own daily health tracking, with a scope I could maintain alone. It is an Expo/React Native and TypeScript app backed by Supabase. I used native integrations for Apple Health, Face ID and reminders, while the backend handles owned records and optional AI calls. The most interesting work was making saves and synchronization recoverable when the phone loses connectivity, without making every feature a large distributed system."

Be ready to draw: the device, local stores/native APIs, Supabase Auth/Data API/PostgreSQL/Storage/Edge Functions and the external provider boundary. Explain one user action all the way through those boxes. "The backend is Supabase" is insufficient because reads, SQL RPCs, Storage calls and Edge invocations take different paths.

Likely challenge: "What did you personally optimize for?" Name personal utility, one-person maintenance and React/TypeScript learning as stated motivations. Explain why reliability still matters for one person. Do not substitute invented user research or claim a public growth target that did not exist. Point to a deliberate limitation, such as targeted offline support, as evidence of scope control.

Why React Native and TypeScript?

Opening: "React Native let me practice React's component/state model and TypeScript while building an iPhone app and a useful browser preview. Shared domain logic and tests reduced duplication, and platform adapters handle capabilities such as HealthKit. I still need native builds and device tests; the browser cannot validate those integrations."

How exactly? Explain `useState` versus refs/effects/context, Expo Router layouts, shared Zod models and `.native`/`.web`/`.ios` adapters. Trace a controlled fluid input into `saveHydration`. TypeScript prevents certain development-time mismatches; it does not validate JSON received over the network.

Why not SwiftUI or Flutter? SwiftUI could simplify Apple-framework integration for an iPhone-only product but would not serve the same React learning/browser reuse goals. Flutter supplies a credible cross-platform Dart stack with its own native-plugin work. The accepted cost of React Native is bridge/dependency/lifecycle complexity. Revisit if native-specific work overwhelms the benefits, not because another framework is universally faster.

Why Supabase rather than writing your own API?

Opening: "Supabase supplies Auth, a PostgreSQL Data API, Storage and server functions, so I can focus on the product's rules. I still wrote ownership policies, constraints and transactional RPCs. External provider secrets stay in Edge Functions. A custom API would give a more independent contract but would add routes, middleware, hosting, monitoring and deployment for a one-person project."

How can the client access the database safely? The client talks to a network Data API with a user token, not an unrestricted PostgreSQL connection. Grants and RLS restrict operations. Sensitive writes go through active-session and owner-aware SQL functions. Service-role server clients need their own careful checks because elevated privileges can bypass ordinary policy enforcement.

What is the lock-in? PostgreSQL data/SQL are relatively portable, but Auth sessions, Storage policies, function deployment and SDK calls are service-specific. A migration would need explicit contracts/data conversion and testing. Do not answer "none" simply because the database engine is open source.

How do you avoid duplicate meals?

Opening: "Before sending a save, I persist an operation ID and the frozen request. The SQL function stores the accepted response in a receipt in the same transaction as the meal rows. If the reply is lost, retrying the same identity/request returns the receipt rather than inserting another meal."

What if the user changes the draft? The runner compares canonical intent and will not reuse an uncertain operation ID with changed content. Resolve the original operation first. Explain why a timestamp/row UUID regenerated on retry would break this property. Name `createMutationRunner`, `createRecords`, `sendMutation` and `commit_health_mutation`.

Why not disable Save or upsert? Button state addresses rapid UI taps; it does not survive termination or prove an uncertain result. An upsert can deduplicate a row identity but can overwrite newer values and does not inherently replay the original multi-row result. A transaction supplies atomic data/receipt commit, while application code chooses the stable identity and replay policy.

What is not covered? Earlier label writes, photo bytes, AI requests and local cleanup do not join that same transaction. Receipt retention is a real storage/lifecycle cost. State the scope before using the phrase exactly once.

What happens when two devices edit the same record?

Opening: "The edit carries the version it was based on. If the database has moved on, the server returns a conflict and keeps the user's edit available for review instead of silently applying last-write-wins. A database lock also orders active writes, but it does not replace the stale-version check."

Concrete example: Both read version 7 at 250 mL. A saves 300 and produces version 8. B's new operation expecting 7 cannot save 200 unnoticed. Replaying A's original operation is different: its receipt returns its already-known result.

Why not merge? Merge independent fields where semantics allow it, as with selected goal baselines. Do not independently merge quantity and derived nutrients or a goal and its basis. The tradeoff is explicit conflicts versus silent loss. Tests should run two separate operations against the same version and inspect the database, not merely assert that an error string was displayed.

What does offline support mean here?

Opening: "Vitals have a durable local cache/outbox, with SQLite on native and IndexedDB on web. A local reading and its pending operation are committed together, then synchronized using stable requests and conflict handling. Other trackers preserve drafts and uncertain saves, but the entire app is not equally offline-capable."

What if an edit happens during sync? The store keeps remote shadow and newer local intent separate. Accepting an older create updates dependent operation versions without replacing the user's newer local edit. Explain `prepareChange`, `acceptChange` and atomic `applyRemotePage`.

Why not queue every request? Features have different dependencies: a meal may need a label, a photo needs bytes and an owned parent, and a goal affects dated interpretation. Broader offline support needs domain-specific conflict/order/cleanup rules. The accepted limitation saves implementation complexity while keeping the most important vitals path durable.

How does Apple Health really work?

Opening: "The iPhone bridge calls HealthKit locally. I request read access for weight, systolic/diastolic BP and heart rate, query anchored weight/BP changes, associate a bounded nearby pulse and map them into owned VitalSample records. Those enter the same local outbox and cloud sync as manual vitals. It is not an Apple web API."

Can you detect denied read permission? Not reliably per type; an empty query can represent denial or no data. Connected is an app choice after the authorization flow. Name `queryQuantitySamplesWithAnchor`, `queryCorrelationSamplesWithAnchor` and `queryQuantitySamples` for nearby pulse.

How do you avoid duplicates and mirror deletion? Stable UUID v5 identities include account, kind and external ID; cached tombstones prevent resurrection. Anchored weight/BP deletions map to tombstones. No general anchored heart-rate deletion stream exists. Anchors advance after successful downstream synchronization, and initial cloud failure currently stops import. Those limitations demonstrate understanding rather than weaken a truthful answer.

Why save nutrition snapshots?

Opening: "A meal should retain what was recorded when it was eaten. I save consumed quantities and nutrient totals in the history row, with optional references to reusable labels/recipes. Editing the reusable label later affects future selection, not old meals."

How does amount editing work? Reconstruct the per-serving ratio from saved totals and serving count. Use the 225-calorie, 1.5-serving oats example to explain two servings becoming 300 without reading a changed label. Unknown optional nutrients stay unknown.

Alternative? Versioned immutable label rows could reduce duplicated fields, but would add version retention, joins and export requirements. Live references are simpler for a catalog but wrong for this chosen historical meaning. Tests verify arithmetic and preservation, not the nutritional truth of a user/provider value.

How do you make AI safe and affordable?

Opening: "The app sends bounded inputs to authenticated Edge handlers with server-held provider keys. The handlers check current permission, reserve quotas and validate output, then return editable estimates or drafts. Models cannot directly write completed health records or choose another user's database identity."

Does structured output prevent hallucination? It constrains format, not truth. Explain refusal/incompletion checks, domain validation, optional training-only history, review and explicit Save. A canceled client wait does not establish canceled provider billing. `store: false` does not mean all provider retention is zero.

What would you improve? If lost expensive responses are common, add generation IDs and retrievable server results with bounded retention rather than blindly retrying. Evaluate real synthetic scenario quality separately from mocked handler tests. Do not claim quotas alone cap all account spending or that code defaults prove live model configuration.

Why are reminders local?

Opening: "Personal routines can be scheduled on the device without a server sending each message. I reconcile saved settings with OS requests and validate notification taps through auth/setup/privacy gates. The tradeoff is device-local preferences and finite windows that need replenishment."

What guarantees delivery? Nothing in the app guarantees visible delivery under all OS conditions. Schedule acceptance, permission and actual display are different. Explain stable IDs/fingerprints, replacement-before-removal, capacity and the 28-day/14-occurrence horizons. A tap opens a validated logger; it never records completion by itself.

Alternative? Server push adds remotely triggered/cross-device control, but also tokens, scheduling workers, push credentials, retries and delivery feedback. It still cannot force an iOS banner. Choose it for an actual product need rather than claiming cloud means guaranteed delivery.

What is your hardest multi-step cleanup?

Opening: "Account deletion destroys the normal session needed to retry, so I persist a continuation capability before preparation. Server cleanup blocks new writes, removes owned objects, deletes Auth/cascaded rows and verifies absence before completion. A hashed-token receipt lets a lost final response be confirmed."

Why not one DELETE? Auth, database and file storage have separate boundaries. Deleting application photo rows does not remove orphan objects. Local tasks can also repersist data if not drained. Explain why the job table survives Auth deletion and why cancellation cannot undo already-started cleanup.

Honest limitation: Offline devices, exported copies and provider retention are separate. This review tested mocks/isolated SQL, not a real deployed deletion. A server worker could improve automatic completion but would need explicit operational recovery.

What would you change for more users?

Opening: "I would first separate account count, concurrent activity and history size, then measure. Likely pressure points are full-history reads/aggregations, local JSON-state rewrites, photo/export memory and provider usage. I would fix paging and refresh gaps, inspect query plans and add targeted observability before adding infrastructure."

How would you know? Give a workload, record rows/bytes/query count, p95 latency, lock waits and memory on representative devices. State assumptions in cost estimates. Explain when a daily aggregate, background export job or narrower lock would help and what correctness complexity it adds.

What is your measured capacity? There is no measured public-user capacity in this review. Account isolation and tests are useful foundations, not load-test results. An honest answer proposes a test rather than inventing a number.

29. Follow-up drills, counterexamples and whiteboard practice

Practice answering each prompt aloud before reading the response direction. For every answer, identify one source path, one invariant, one failure window and one test. If you cannot explain the sequence without its labels, reread the linked chapter rather than memorizing another sentence.

Retry and concurrency drills

The same operation ID arrives with different meal contents. What happens? The server compares the saved request and rejects identity reuse with a changed payload. The client also protects a pending slot from changed intent. Returning the old response without comparison would falsely confirm the new contents. Creating a new operation silently could duplicate the originally uncertain meal. Review Chapter 9.

The response is accepted, then the app terminates before clearing the draft. Completed pending data remains until draft cleanup, allowing replay of the local result/receipt instead of a new insertion. Name the order: save draft, freeze request, commit server result, mark completed, clear draft, clear completed pending state. Reversing cleanup steps needs separate analysis.

Two distinct meals have identical contents and time. Should the server deduplicate them? Not merely because their values match. They may be two legitimate events. An operation ID identifies intent; content equality across different IDs does not establish a retry. A database uniqueness rule on food/time could reject valid logs.

A server process fails after changing rows but before inserting the receipt. If those statements are inside the same PostgreSQL transaction and it does not commit, they roll back together. If code moved the receipt into a second transaction, the duplicate-prevention guarantee would break. Explain the boundary rather than relying on a generic "database is reliable" claim.

You have locks, so why can an edit still conflict? The user edited a previously read version outside the transaction. The lock only serializes the commit attempt. A version comparison checks whether the premise of the edit remains current. These mechanisms are complementary.

Could a plain global sequence replace the per-account cursor? It can allocate values out of commit order: a lower-numbered transaction can commit after a higher one was consumed. Sustain's locked counter avoids that particular hole. The current mutable-row stream still is not an immutable event log. Review Chapter 10.

What if receipts are purged after a day? A phone offline longer than a day could retry an accepted operation whose receipt is gone. Reserved row/deletion IDs may reject some cases, but the original result-replay contract is no longer generally available. Define retention and client retry limits together before purging.

Identity and security drills

A caller changes `p_user_id` to another account. The active-session/account check must reject it independently of the field's syntax. Owner-aware foreign keys and RLS protect additional boundaries. Client `eq` filters are not enough because the caller can omit them.

A set uses the caller's `user_id` but another user's `session_id`. A simple owner policy on the child could allow the child owner while missing the parent mismatch. The composite owner/parent foreign key closes that relationship gap. Draw the two columns on both sides.

A signed access token has not expired, but the user signed out everywhere. Signature/expiry alone may not capture revocation. Explain `require_active_session` and its session identifier check. Do not claim every provider endpoint in the ecosystem automatically follows Sustain's custom check.

A public key is visible in the binary. Is that a leak? Intentionally public project configuration is expected to be visible. A service-role or provider key would be a serious different exposure. Explain privilege, not just whether a string contains the word key.

The privacy cover hides the screen. Are local drafts encrypted? The cover restricts presentation/access, while ordinary draft storage has its own protection. Keychain-protected biometric secrets do not imply all local data uses equivalent encryption. State the actual store per asset.

A user denies heart-rate access but grants weight. HealthKit can return partial/empty data without disclosing per-type read denial. The importer can show available readings and preserve uncertainty. Do not mark missing pulse as zero or claim the entire integration is unauthorized.

Asynchronous UI and data-model drills

September returns after the user selects October. A month/account-specific resource key and stale-publication guards prevent the old result replacing the selected month. Canceling a request is useful but not sufficient if the response already arrived or cancellation is unsupported. Review Chapter 17.

Why use a ref for a submission guard if there is saving state? A ref updates synchronously for code coordination without waiting for a render. Saving state makes the UI reflect it. Neither alone survives process death; durable pending state is a separate mechanism.

A reusable food label is corrected. Should old meals change? Under Sustain's snapshot semantics, no. New selections can use the corrected basis. Old amount edits use the old saved ratio. A product wanting retroactive correction would need an explicit operation and history/audit policy.

No sodium value is provided. Why not save zero? Zero is a known value and can be summed confidently; unknown means the total may be incomplete. Preserving that distinction prevents false precision in recipes/exports. A schema's optional field is a semantic choice, not just permissive typing.

A user logs near midnight while traveling. Which day is it? Distinguish the stored UTC instant, the user-selected local entry day and the timezone used for current grouping. Explain the actual entry-date helper and local-midnight query boundaries. Do not claim a timezone-free date is automatically an instant.

A chart average is a value never directly measured. Is that wrong? It is valid if labeled/understood as a bucket statistic and raw readings remain accessible. Smoothing must not invent apparent extremes. Explain deduplication, pairing, averaging and interpolation as separate stages.

External-effect drills

A photo upload succeeds and metadata rejects the fourth photo. Current source can leave an orphan. Propose compensation/staging with retries; do not claim the SQL transaction removed the object. Test actual Storage and metadata independently. Review Chapter 19.

The user cancels AI after the provider starts. The app should avoid applying a late result, but provider work/quota/billing may already exist. A generation ID/result store could support retrieval; cancellation needs clearly defined semantics rather than a promise of rollback.

A notification is registered, then permission is disabled in Settings. Saved configuration and OS delivery diverge. Reconciliation/permission feedback must detect the state when the app runs. Server push would still depend on permission and OS behavior.

Deletion removes Auth, then the final response disappears. The continuation token and surviving completion receipt permit status recovery. An Auth-cascaded job record would disappear at exactly the point it is needed. Review Chapter 21.

An export returns HTTP success but only 500 rows. Transport succeeded; completeness may have failed if code assumes a 1,000-row page. Demonstrate with a capped fixture and compare exported identities against expected data. Error-free HTTP is not enough.

Architecture and operations drills

Why not a cache server immediately? First identify repeated expensive work, acceptable staleness, owner scope and invalidation. The existing resource/catalog caches already solve specific needs. A new cache adds another failure/state system and can amplify privacy errors if scoped incorrectly.

Why not split each feature into a microservice? A single-owner system can use module boundaries without separate deployments. Service boundaries would add remote contracts, retries and observability. Consider a split when independent team ownership or measured scaling/isolation requirements outweigh that cost.

What if a migration and old installed app disagree? Treat the old app as a client contract. Prefer compatible expansion before removal; when enforcing a security change requires rejection, provide clear update-required recovery. Reverting source does not undo accepted database changes.

What does a passing test count prove? Only the scenarios and assertions in those tests. Name the 110/112 browser result and native/hosted limitations. Explain why SQL stubs, mocked providers and browser adapters need integration/device evidence rather than presenting all tests as production simulation.

Which guarantee would you remove to simplify? Identify a low-risk local preference where last-write-wins/direct storage is sufficient. Do not remove owner checks or retry protection from completed health records merely because the original user count was one. Complexity should track consequence and failure likelihood.

Three whiteboard exercises

Exercise 1: save a meal across a lost reply. Draw memory, AsyncStorage, network, SQL transaction and receipt. Number each handoff and mark the point where the server knows success but the client does not. Show exactly what stays unchanged on retry. Add a changed draft and explain why it cannot overwrite the old operation identity.

Exercise 2: import a BP pair with pulse. Draw authorization quantity types, anchored correlation query, component extraction, unit conversion, nearby pulse query, deterministic IDs, local queue and cloud cursor. Mark the lack of a general heart-rate deletion stream and the distinction between HealthKit anchors and Supabase cursors.

Exercise 3: redesign export for ten years of photos/history. Start with the current memory/pagination boundaries. Propose robust paging first, then decide whether snapshot consistency and background generation are requirements. If introducing a job, specify ownership, status, resumability, result expiry, temporary file cleanup and how failure is tested. Do not jump straight to a product name such as Redis.

A method for questions this guide did not predict

Restate the requirement, ask which failure/scale assumption matters, identify the invariant and trace the data boundary. Compare the simplest workable alternative with the current design. State what would be measured or tested before committing to an improvement. If you do not know an exact platform detail, say what the source establishes and what documentation/experiment would answer it.

A useful answer structure is: "Today the code does X, because requirement Y matters. It guarantees Z within this boundary. Failure F is handled this way, while G is still a limitation. Alternative A would change these responsibilities. I would choose it if measurement or product requirement B justified the added cost." Fill each slot with concrete Sustain facts, not generic adjectives.

You are ready for a deep dive when you can explain a feature without the guide, then defend it under a changed assumption. Being prepared does not mean claiming certainty about untested behavior. Precise limits, a plausible alternative and a test that would settle the question are themselves strong engineering answers.

30. Working glossary

These definitions reinforce the chapters. They are intentionally tied to the project so the terms remain mechanisms rather than vocabulary to memorize.

Account isolation. Keeping one user's rows, local keys and asynchronous results separate from another's. RLS handles cloud row boundaries; client guards and key design handle different local risks.

Adapter. A small implementation that connects a shared responsibility to a specific platform/service. SQLite and IndexedDB adapters preserve a common vitals-store purpose with different APIs.

Anchor / cursor. A position used to continue reading changes. HealthKit supplies opaque anchors; Sustain's cloud vitals stream uses a per-account numeric change cursor. They are not interchangeable.

API / SDK / RPC. An API is a callable contract. An SDK is a library that implements repeated service-protocol work. An RPC, remote procedure call, requests execution of a named remote function, such as a PostgreSQL function through Supabase.

Atomicity. All changes within a defined transaction happen together or roll back. Meal rows plus their receipt share a database boundary; photo bytes do not join it.

Authentication / authorization. Establishing identity versus permitting an action. A valid login does not authorize reading another account's rows.

Bearer token / capability. A secret whose possession authorizes defined access. Access tokens, signed photo URLs and deletion continuation tokens have different scope/lifetimes and must not be treated as public identifiers.

Cache / invalidation. A retained copy for faster/reusable reads, and the act of marking it in need of refresh. Invalidation is not itself a completed database write.

Canonical representation. A predictable representation for comparison. The mutation runner orders object keys so equivalent intent can be compared without depending on incidental property order.

Constraint / foreign key. A database-enforced rule and a specific rule requiring a referenced parent. Composite owner/parent keys prevent cross-account child relationships.

Context / hook / ref. React mechanisms for sharing values, attaching component behavior and retaining mutable non-rendering state. None inherently provides durable persistence.

Durability. Retention of an acknowledged write under the store's contract. It differs from a draft merely existing in memory and from an independently verified backup/restore policy.

Edge Function. Hosted server code used here for protected provider access and account/device operations. It is not the path for every ordinary table read.

Idempotency / receipt. Repeating the same identified operation without repeating its effect, implemented here with saved request/response records plus a transaction.

Index / query plan. Maintained lookup structure and the database's chosen execution strategy. An index's presence does not prove a particular query uses it efficiently.

JSON / JSONB / JWT. JSON is a structured text format; JSONB is PostgreSQL's structured JSON storage type; JWT is a signed token format. Sharing the letters JSON does not make them the same mechanism.

Keyset pagination. Reading the next page after the last ordered key. It avoids some offset problems but does not inherently create a snapshot across multiple requests.

Migration. An ordered executable schema/data change. The set in the repository and the set applied to a hosted project are separate facts to verify.

Optimistic concurrency / version. Allowing editing without holding a long transaction, then checking that the stored version still matches the edit's basis before accepting it.

Outbox / pending mutation / draft. A durable queue of local changes, a saved identity/request awaiting confirmation or cleanup, and unfinished user input. Sustain uses all three with different semantics.

RLS / grant / role. Row-level security chooses permitted rows; a grant allows operations; a role is the database execution identity. Privileged function execution can change which checks must be explicit.

Snapshot. Either copied historical values, such as consumed nutrition, or a point-in-time read view. Those meanings differ: meal snapshots do not make a multi-table export transactionally consistent.

Tombstone. A retained deletion marker that lets other devices learn a record was removed and prevents accidental resurrection of its identity.

Transaction / lock / isolation. A commit/rollback boundary, coordination of competing operations and the visibility/order rules for overlapping transactions. A version check still serves a distinct stale-edit purpose.

Validation. Checking shape, type, range and domain rules at runtime. It does not prove authorization, provider truth or medical appropriateness.

p95 / RPO / RTO. A latency percentile, acceptable recovery data-loss interval and acceptable recovery duration. They require explicit measurements/objectives; none is inferred from a passing test count.

31. Source map, evidence and verification boundaries

The repository is `HealthApp`, reviewed on `main` at `4fb1bddb1d95af71564828cc3cf8d202bf12ae7b`. Paths below are relative to that repository. For readability, **M** means `apps/mobile`; **F** means `supabase/functions`; **SQL** means `supabase/migrations`. Function names provide stable navigation targets within those files. The accompanying evidence manifest records file/log hashes so the snapshot can be distinguished from later edits.

C01 - Scope and review record

`AGENTS.md`; `docs/STATUS.md`; `docs/ROADMAP.md`; `docs/IMPLEMENTATION_LOG.md`; `docs/IMPLEMENTATION_REVIEW_2026-09-25.md`. These establish repository workflow, the active physical-device item, historical deployment statements and this review's results. Historical statements are not new hosted attestations.

C02 - Screens, navigation and composition

`M/app/_layout.tsx`; `M/app/(app)/_layout.tsx`; `M/app/create.tsx`; `M/src/features/auth/entry-provider.tsx` (`EntryProvider`, `ContinueEntry`); `M/src/ui/appearance.tsx`; `M/src/ui/motion.tsx`. Read the provider/layout order before individual screens. Shared UI source supplies controls and styling; actual route reachability determines current product scope.

C03 - Email authentication and session events

`M/app/(auth)/sign-in.tsx`; `M/app/(auth)/sign-up.tsx`; `M/src/features/auth/auth-provider.tsx` (`AuthProvider`, `useAuth`); `M/src/features/auth/entry-state.ts`. Follow SDK signup/sign-in and callback handling into provider events and protected routing, rather than interpreting the returned user object alone as successful entry.

C04 - Client configuration and protected local storage

`M/src/lib/supabase.ts`; `M/src/lib/config.ts`; `M/src/lib/secure-store.ts`; installed `node_modules/@supabase/auth-js/src/GoTrueClient.ts` and `@supabase/postgrest-js/src/PostgrestClient.ts` for SDK-generated endpoint/method behavior, with dependency identity in `package-lock.json`. Inspect `persistSession`, token-refresh settings, legacy cleanup and the native/browser storage distinction. No configuration secrets are reproduced in this guide.

C05 - Client mutation protocol and refresh events

`M/src/lib/mutation-model.ts` (`createMutationRunner`, canonical intent and pending state); `M/src/lib/mutations.ts` (`sendMutation`, `createRecords`, `changeRecord`, `completePendingDraftSave`, `retryPendingMutation`, `assertAccount`); `M/src/lib/data-changes.ts`. These establish client replay, validation/error classification, photo handoffs and change notifications.

C06 - Database construction and schema evolution

All 37 ordered SQL migration files from `202608290001_release_1_vitals.sql` through `202609250001_privacy_controls.sql`. Particularly important foundations are `202608290002_training_nutrition.sql`, `202608310001_food_catalog.sql`, `202609020001_tracking_expansion.sql`, `202609170001_food_recipes.sql` and `202609170002_drink_categories.sql`. Later migrations alter grants/types/relationships; the initial definition alone is not the effective schema.

C07 - Database write and synchronization enforcement

`SQL/202609180003_biometric_hardening.sql` (`require_active_session`); `SQL/202609180004_consistent_writes.sql` (`commit_health_mutation`, `read_vital_changes`, `bump_record_version`, `assign_vital_cursor`, receipts/deleted IDs/counters); `SQL/202609180005_remaining_ownership.sql`. These are the authoritative transaction, version, cursor and ownership boundaries discussed in Chapters 6-10.

C08 - Onboarding and settings continuity

`M/src/features/onboarding/repository.ts`; `M/src/features/onboarding/provider.tsx`; `SQL/202609210001_account_onboarding.sql` (`save_account_setup`, `create_account_setup`, setup receipts). Read pending-write restoration and established-account backfill together with current setup fields.

C09 - Face ID enrollment and exchange

`M/src/features/auth/biometric-auth.ts` (`getFaceIdAvailability`, `saveFaceIdLoginCredential`, `signInWithFaceIdCredential`, `waitForBiometricForeground`); `biometric-credential.ts`; `F/biometric-auth/index.ts`; `F/_shared/biometric-device.ts`; `F/_shared/biometric-enrollment.ts`; biometric migrations 202609040001 and 202609180003. The source binds device/secret/account and separates privacy unlock from sign-in.

C10 - Local vitals and cloud synchronization

`M/src/features/vitals/store-model.ts`; `sqlite-store.ts`; `storage.native.ts`; `storage.web.ts`; `sync.ts` (`syncVitals`, `queueLocalVitals`, `markVitalsDeleted`, `applyRemotePage` via storage). `M/src/domain/vitals.ts` supplies chart/pairing domain behavior. Read the operation dependency and remote-shadow rules before claiming what an accepted response can replace.

C11 - Apple Health integration

`M/src/features/healthkit/reader.ios.ts` (`healthKitAvailability`, `requestHealthKitAuthorization`, `readHealthKitData`, `pulseNearBloodPressure`); `permissions.ts`; `identity.ts` (`healthKitRecordId`); `mapping.ts`; `sync.ts` (`connectHealthKit`, `importHealthKitData`); `state.ts`; `M/app.json`. Mapping helpers not called by the active reader are not treated as implemented imports.

C12 - Training and cardio

`M/app/(app)/workout.tsx`; `M/src/features/training/repository.ts` (`saveWorkout`, `replaceWorkout`, `getExerciseGuidance`); training domain/model and draft modules; `use-history-data.ts`. Related SQL is the training migration chain plus the hardened mutation function. The waveform of UI refresh must also be checked against C05/C18.

C13 - Nutrition, snapshots, drafts and recipes

`M/app/(app)/nutrition.tsx`; `M/src/features/nutrition/model.ts`; `repository.ts` (`saveNutritionMeal`, `saveFoodProfile`, `getFoodSuggestions`, `getFoodHistory`, `deleteFood`, `saveFoodRecipe`); `draft.ts`; `recipe.ts`; `food-editor.tsx`; `recipe-manager.tsx`. These define unit bases, immutable consumed fields, label-before-meal writes and recipe yield arithmetic.

C14 - Barcode and provider normalization

`F/resolve-food-barcode/index.ts`; `F/_shared/barcode-normalization.ts`; `F/_shared/food-normalization.ts`; the client's food repository/model and camera configuration. These establish the actual Open Food Facts endpoint, timeout/cache decisions and normalized return shape.

C15 - AI meal pipeline

`M/src/features/nutrition/ai-meal-client.ts` (`estimateMeal`); `ai-meal-editor.tsx`; `ai-meal-image.ts`; `ai-meal.ts`; `F/estimate-meal/handler.ts` (`createMealHandler`); `index.ts`; `F/_shared/meal-estimate.ts`; `F/_shared/ai-privacy.ts`. The index's source default is `gpt-5.4-mini`, overridable by server configuration; this is not a deployed-model attestation.

C16 - Workout planner

`M/src/features/coach/repository.ts` (`sendCoachMessage`, profile/action operations); `draft-actions.ts`; `draft-action-model.ts`; `F/coach-chat/handler.ts`; `index.ts`; `F/_shared/coach-context.ts`; `coach-action-validation.ts`; `workout-planning.ts`; coach migrations including `save_coach_turn`. Source defaults are `gpt-5.6-luna` and `gpt-5.6-terra`, subject to server overrides and current handler/UI routing.

C17 - Fluids and goal helpers

`M/src/features/hydration/model.ts`; `repository.ts` (`saveHydration`, `getTodayHydrationTotals`, preset operations); `categories.ts`; `F/_shared/hydration.ts`; `M/src/features/goals/calculator.ts` (`calculateBmr`, `calculateMaintenance`, `calorieTargetForLossRate`, `calculateFluidGoal`); goal repository/helper-model/helper-draft modules. Calculations describe product behavior rather than clinical validation.

C18 - Summary, chart calculations and streaks

`M/src/features/summary/use-summary.ts`; `data.ts`; `dashboard.tsx`; `layout.ts`; `storage.ts`; `training.ts`; `meals.ts`; `M/src/lib/use-resource.ts`; `resource-store.ts`; `M/src/domain/vitals.ts`; `M/src/features/streaks/engine.ts`; `repository.ts`; `SQL/202609180001_summary_streaks.sql`; `202609220001_summary_refinements.sql`. These establish resource keys, source reads, derived values and effective-date behavior.

C19 - Reminder schedule construction and reconciliation

`M/src/features/reminders/model.ts`; `routine-model.ts`; `schedule-model.ts`; `service.ts` (`saveRoutinePreferences`, `saveCustomConfiguration`, `refreshReminderSchedules`, `restToday`); `device.native.ts` (Expo Notifications adapter); reminder repository. The native adapter and pure schedule model have different verification boundaries.

C20 - Notification capture and protected destinations

`M/src/features/reminders/notification-observer.tsx`; `navigation-model.ts`; `device.native.ts` (`observeNotificationTaps`); `M/src/features/auth/entry-provider.tsx`; `entry-state.ts`; `docs/decisions/ADR-010-sustain-entry.md`. These explain startup capture, account/expiry checks, serialized intents and routing through setup/privacy gates.

C21 - Progress photos

M/src/features/progress-photos/image.ts (selectProgressPhoto, processing); model.ts; repository.ts (uploadProgressPhoto, getProgressPhotos, deleteProgressPhoto); M/src/features/privacy/jpeg.ts; photo-cache.ts; C05; SQL/202609040002_progress_photos.sql; 202609160001_progress_photos_per_entry.sql; ownership hardening. Current sender code, not an older compensation comment, establishes actual orphan behavior.

C22 - Portable export

M/src/features/data-export/repository.ts (loadRows and account dataset collection); local-inventory.ts; model.ts (csvValue, exportTextFiles); archive.native.ts; archive.web.ts (shareHealthDataExport). Compare the native streaming output with the web buffer/Blob path and the shared dataset-memory/pagination limits.

C23 - Deletion and privileged cleanup

M/src/features/auth/account-deletion.tsx (requestDeletion, resumeDeletion, AccountDeletionGate, DeleteAccountSection); F/delete-account/index.ts; cleanup.ts; SQL/202609220002_account_deletion.sql; 202609250001_privacy_controls.sql. These establish preparation, capability hashing, write guards, bounded file cleanup, Auth deletion, completion and local recovery.

C24 - Privacy controls and release preparation

M/src/features/privacy/; M/plugins/with-private-backups.js; legal/policies.json; legal/release.json; docs/PRIVACY_RELEASE_REVIEW.md; PRIVACY_APPLICABILITY.md; PRIVACY_DATA_AND_OPERATIONS.md; privacy migration 202609250001. Policy/operator/provider fields remain unresolved; documentation is not legal certification.

C25 - Documented decisions

docs/decisions/ADR-001-hosted-supabase-without-docker.md through ADR-010. These record specific design decisions and later amendments. Source is authoritative for current behavior where historical text differs, including narrowed AI planning, photo cleanup and current storage/security behavior. Alternatives in Chapter 23 are labeled retrospective analysis rather than invented historical deliberation.

C26 - Build, deployment and CI

package.json; M/package.json; M/app.json; M/app.config.ts; M/eas.json; .github/workflows/ci.yml; Supabase function configuration and migration files; scripts/build-legal.mjs. Production gating, updates disabled, native variants and CI commands are repository facts; no new build/deploy was made for this document.

C27-C29 - Supporting implementation evidence

C27: All SQL indexes, grants, policies, constraints and function bodies in the ordered migration chain, with the maintained harness in `scripts/check-onboarding-sql.mjs` and regressions in `supabase/tests/`. **C28:** Application `*.test.ts` files, server shared/handler tests and `M/e2e/` fixture/specs; read assertions and mocks before citing coverage. **C29:** `M/src/ui/`, appearance/motion/shared-sheet controls and privacy boundaries, plus accessibility/native preparation described in C24. Source files are hashed in the companion manifest; inclusion in that inventory does not mean every possible branch was executed.

C30 - Checks executed and retained logs

`dist/pre-guide-review-check-authorized.log`: lint/TypeScript, 256 app and 70 Edge tests pass. The initial sandbox run could not spawn workers; the authorized rerun succeeded. `dist/pre-guide-review-full-sql.log`: all 37 repository migrations and maintained SQL regressions pass. `dist/pre-guide-review-sql.log`: older harness fails on its missing Storage `owner_id` stub.

`M/dist/pre-guide-review-browser.log`: 110/112 pass. `M/dist/pre-guide-review-browser-targeted.log`: the same two cases fail again. Failure locations are `M/e2e/design.spec.ts:18` and `M/e2e/interaction-revisions.spec.ts:216`; screenshots/error context remain in the corresponding `pre-guide-e2e-results` and `pre-guide-e2e-targeted-results` directories. These fixtures are synthetic and do not mutate hosted user data.

PDF QA is separate from app testing: the final guide is paginated, rendered, text/links/outline checked and visually inspected. The machine-readable layout/evidence records live with the local authoring artifacts. App tests were not rerun merely for prose edits because application behavior did not change.

Official references checked for authoring

These sources explain platform behavior or deployment concepts. They do not establish which Sustain build or server version is deployed. URLs were checked on 25 September 2026; time-sensitive requirements should be checked again at release. No quoted service prices or unsupported current throughput limits appear in the report.

R01 - Database migrations. Supabase migration workflow. Ordered schema changes and deployment tracking; used in Chapters 5 and 24.

R02 - Row-level security. Supabase PostgreSQL RLS. Row policies and authenticated access; used to distinguish the platform mechanism from Sustain's own policies/functions.

R03 - Native development builds. Expo development-build introduction. Native modules and the distinction from Expo Go; source configuration determines Sustain's specific dependency set.

R04 - HealthKit read-permission privacy. Apple HKAuthorizationStatus documentation. Explains why read denial is not revealed as a dependable per-type status. Used in Chapter 11.

R05 - Structured model output. OpenAI Structured Outputs. Schema-constrained output format; Sustain's own handler/model validation and editable review provide additional boundaries.

R06 - Provider data controls. OpenAI data controls. Distinguishes response application state from other retention controls; `store: false` is not represented as universal zero retention.

R07 - Beta distribution. Apple TestFlight overview. Beta distribution and review context. No fixed future tester/build limits are asserted.

R08 - Edge Function deployment. Supabase function deployment. Function release is distinct from schema migration and mobile distribution.

R09 - React state. React: State, a component's memory. The component-memory/render relationship; current application examples come from Sustain, not copied tutorial code.

R10 - Flutter architecture. Flutter architectural overview. Framework/native integration context for a credible alternative, not a measured comparison with Sustain.

R11 - SwiftUI. Apple SwiftUI documentation. Native declarative UI alternative; tradeoff analysis is retrospective.

R12 - Transactions. PostgreSQL transaction tutorial. All-or-nothing transaction foundations; actual boundaries/locks are defined by Sustain's SQL.

R13 - Client environment values. Expo environment variables. Client-visible environment substitution and why public bundle values are not secret storage.

Final teaching audit

For each major feature, this guide identifies the user purpose, participating runtime, source path, representative inputs/storage, execution order, visible update, failures and alternatives. Reliability chapters add before/after state and transaction/retry distinctions. The interview chapters require explaining those mechanisms under changed assumptions. Remaining unknowns are named rather than filled with inferred deployment, medical, security or scale claims.

The intended outcome is a reconstructable model of the application: you can start at a control, name the next call, locate the durable state, describe the permission boundary and explain what the user sees when the next step fails. That model supports deeper questioning more effectively than a list of technologies or an unqualified claim that the system is secure and scalable.